Meta Global AI Developer Hackathon 2026 — 기술·설계 조사 보고서

작성 2026-10-28 (KST) · 주식회사 솔버린 · Lookout 팀 내부용 범위: Meta 모델과 에이전트 설계 방식, 모델 학습(포스트트레이닝), 사용자별 토큰·비용 한도, 에이전트 동작 패턴, 해커톤이 추구하는 아키텍처·모델 활용, 공식 코드·예제·파트너 자료. 모든 항목에 출처 링크를 붙였다. 본문은 종합이고, 부록 A·B·C에 페이지별 상세 노트를 그대로 실었다.

---

0. 한 장 요약

1. **Meta가 이 해커톤에서 밀고 있는 것**: Muse Spark(또는 오픈 웨이트)를 두뇌로 삼고, **감사 가능(auditable)·검증 가능(verifiable)한 하네스**로 감싼 **장기 실행 에이전트**. 공식 문서·쿡북·Muse Code·라이브 세션(Agents 101 "Build a multimodal AI agent", Agents 102 "Best practices for verifiable agents")이 일관되게 agent loop · tool calling · multi-agent orchestration · computer use · staged approvals · audit log를 가르친다.
2. **설계 기본값**: Responses API(`POST /v1/responses`) + function tools + `previous_response_id` 로 턴 간 추론 유지. Chat Completions는 턴마다 추론을 처음부터 해서 에이전트 루프에 부적합. 모델 분담은 Wonder가 메인 루프, Spark가 서브에이전트·저비용 작업.
3. **모델 학습**: Meta Model API에는 호스티드 파인튜닝이 없다. 오픈 웨이트(Glimmer 30B, Spark Open 816B)를 자기 GPU(Axolotl·TRL·TorchTitan)나 파트너(Fireworks serverless LoRA, Together SFT/GRPO)에서 학습한다. 해커톤 규모에서 현실적인 건 Glimmer QLoRA(H100 1장) 또는 Fireworks serverless LoRA.
4. **사용자별 토큰 한도**: API에 없다. 레이트 리밋은 **팀 단위**(Spark 3,000 RPM / 4M TPM)로 모든 키가 공유하고, 지출 한도·프로젝트 스코프·키별 한도 설정은 문서에 없다. 앱이 서버 프록시 뒤에서 사용자별 예산 원장·토큰 버킷· `x-ratelimit-*` 헤더 기반 스로틀을 직접 구현해야 한다.
5. **Lookout에 주는 시사점**: 이미 설계한 것(원장, 정책 게이트, 멱등 실행, 승인 큐)이 Meta가 보상하는 패턴과 정확히 겹친다. 더할 것은 (a) Responses API + `previous_response_id` 로 사건 하나를 한 대화로 유지, (b) 구조화 출력으로 Action[] 생성, (c) Spark Open(Baseten `meta-models/muse-spark-open-1026`) 경로를 폴백으로 붙여 Open Weight 보너스 자격 확보, (d) 사용자별 비용 원장을 "신뢰 레이어"에 포함.

주의: 서드파티 전사본(meta-ai-hackathon/meta-hack)에는 심사 가중치가 Impact 30% / Meta tooling 25% / Demo 25% / Technical depth 20%로 적혀 있다. 공식 Discord `#faqs` 는 "scored equally"라고 했다. 참가자 가이드(로그인 필요) 원문으로 확인해야 한다.

---

1. 해커톤이 요구하고 보상하는 것

항목	내용	출처
테마	Build for Your World("Bring a problem you know firsthand") / Push the Frontier	행사 페이지, Meta 이벤트 페이지
트랙 심사 문구	Startup: "Judged on technical execution and whether it holds up in production." Indie: "creativity, technical ambition, and whether it works."	행사 페이지
관문	테마 적합 + Meta Model API 또는 오픈 웨이트 실사용. 둘 중 하나라도 실패하면 실격	Discord `#faqs` , meta-hack 전사본
본 심사	Impact / Use of Meta AI tooling("going beyond basic integration") / Technical depth / Demo quality	Discord `#faqs` (동일 가중), meta-hack 전사본(30/25/25/20)
보너스	Most Viral $20K(X 게시), Open Weight $10K(Glimmer/Spark Open 자체·파트너 추론), Community Vote $10K, Best Agentic Use-Case $5K("strongest agent autonomy and multi-step workflows"), Best Creative Build $5K	Discord `#faqs`
제출	11/4 21:00 ET. 3분 영상(주 심사물), 공개 저장소, 기여자 전원, 행사 중 만든 것만 시연	Discord `#rules`
제공 도구	Muse Spark 시리즈, Muse Wonder, Muse Image, Voice Transcribe, SAM 3.1, 오픈 웨이트(Spark Open, Glimmer), Muse Code	행사 페이지
크레딧	Meta $150/인, Baseten $100 + 2M TPM(용도 심사), Modal $100, Fireworks $100(행사 페이지) vs $25(전사본, 불일치)	행사 페이지, Discord `#faqs`
Meta 발표 문구	"Build on Meta Model API, Muse Code or our open-weights models." / "Build the future of superintelligence with us"	Connect 2026 recap, X @MetaforDevs

라이브 세션(녹화는 Discord `#resources` 에만, 공개 웹에 없음): 10/27 Agents 101 — Suraj Subramanian "Build a multimodal AI agent" · 10/28 Agents 102 — Yash Khare "Best practices for verifiable agents" · 10/30 Fireworks — Sinan Ozdemir "Specialized intelligence on Fireworks". 공개 대체 자료: Building with Muse Spark and Muse Code (Connect 2026).

과거 Meta 해커톤 우승 패턴 (LlamaCon 2025, Llama 3 2024): 실제 도메인 데이터(사내 문서, CCTV, 지질 보고서)에 모델을 꽂아 사람이 매일 하는 반복 업무를 대체 · 그 릴리스의 차별 기능(멀티모달, 롱컨텍스트)을 눈에 띄게 사용 · 물리 세계 연결(글래스, 카메라) · 멀티에이전트(토론+심판).

---

2. 모델 목록과 강점 (요약)

상세는 docs/meta-models.md.

모델	강점	가격	제약
Muse Wonder 1.0 (`muse-wonder-1.0`)	가장 강함. 장기 에이전트, 서브에이전트, 병렬 도구, 컴퓨터 사용 주 모델, PDF·영상·오디오, 1M ctx	$4 / $20 per 1M	preview. **한국 미제공**(API·Muse Code 모두). 지역 통제 우회 금지
Muse Spark 1.3 (`muse-spark-1.3`)	에이전트 주력. 병렬·스트리밍 도구 호출, 구조화 출력, 검색 그라운딩, PDF·영상, 컴퓨터 사용(저비용), 1M ctx	$1.25 / $4.25 per 1M, Contributor $0.10 / $0.20	Contributor는 콘텐츠 학습 동의
Muse Image 1.0	생성+편집, 멀티턴, 참조 이미지 1-10장, 자체 웹 검색·코드 실행	$0.01/장	인도 미제공
Muse Voice Transcribe	한국어 포함 25개 언어, 화자 분리, 키워드 바이어싱	$0.18/시간	턴 단위 타임스탬프만, 배치 10분·32MB
SAM 3.1	명사 프롬프트 분할, 영상 추적	$0.0025/장, 영상 1천 프레임 $0.20	요청당 개념 하나
Muse Glimmer (오픈, 30B)	로컬·엣지 에이전트. Q4 GGUF 17GB(≈19GiB), Ollama MLX로 Mac	0 (Together $0.35/$1.50)	턴당 도구 호출 1개, 병렬 불가. reasoning 끌 수 없음
Muse Spark Open (오픈, 816B MoE)	장기 에이전트용 오픈 웨이트. Baseten·Fireworks·Together·Groq·Modal 호스팅	호스팅 요금	자체 서빙은 B300급

출처: Models, Pricing, Open-weight models.

---

3. 에이전트 설계 방식 — Meta 문서가 권장하는 아키텍처

3.1 프로토콜 선택

세 포맷(Responses / Chat Completions / Messages)은 같은 모델·인증·가격. **새 작업의 기본은 Responses.** 턴 간 추론 연속성, search grounding, tool search, background 실행, 파일 입력은 Responses에서만 전부 된다. Chat Completions는 외부 키에서 `reasoning_content` 가 비워져 턴 간 추론이 끊긴다. Messages는 Claude Agent SDK·Claude Code용(stateless, `thinking` replay). — Protocols, Reasoning

3.2 도구 호출

- 모델은 `function_call` 만 내고 실행은 앱이 한다("The model does not execute developer-defined tools itself"). 결과는 `function_call_output` + `previous_response_id` 로 반환.
- `tool_choice` 는 `"auto"` 만. `"required"` ·함수 지정은 400 → "반드시 도구 호출"은 프롬프트·루프 설계로.
- `parallel_tool_calls` 기본 true. `strict: true` 라도 인자 검증은 코드에서.
- 내장 도구: `web_search` (검색 그라운딩), `tool_search` (지연 로딩). — Tool calling

3.3 도구가 많을 때: tool search

`{"type":"tool_search"}` + 함수에 `defer_loading: true` . 이름·설명만 노출, 스키마는 필요할 때 로드되어 컨텍스트 끝에 붙어 캐시를 보존. `namespace` 로 묶고 네임스페이스당 10개 미만. 한번 로드한 세트는 바꾸지 말 것. `json_schema` 응답 포맷과 동시 사용 불가. — Tool search

3.4 구조화 출력

JSON Schema로 디코딩 자체를 제약. 추출·분류·Action 생성·자기평가에 쓴다. Chat Completions는 `response_format` , Responses는 `text.format` . 재귀 스키마 불가, 깊이 10, `strict` 는 서버 검증 여부만. — Structured output

3.5 추론 강도와 비용

`reasoning_effort` : minimal / low / medium / high / xhigh / max(`"none"` 은 400). 추론 토큰은 출력 토큰으로 과금되고 `max_output_tokens` 에 포함 → 예산을 넉넉히. 멀티턴 추론은 `previous_response_id` 또는 `include: ["reasoning.encrypted_content"]` replay. — Reasoning

3.6 프롬프트 캐싱

자동 prefix caching. 안정 부분(system prompt·지시·예시)을 앞에, 변하는 부분을 뒤에. `cached_tokens` 로 모니터링, 고볼륨은 `prompt_cache_key` , 버스트는 `prompt_cache_retention: "24h"` . 캐시 입력 Spark $0.15. — Prompt caching

3.7 컴퓨터 사용

`tools=[{"type":"computer"}]`, Responses 전용. 모델은 액션 프로토콜(click/type/scroll/…)만 내고 브라우저·드라이버·스크린샷·안전 처리는 앱이 제공("defines the action protocol, not an environment"). `pending_safety_checks` 는 사람의 결정 지점. 일회용 VM/컨테이너 권장. Wonder 기준이지만 Spark로 같은 루프를 저비용 실행. — Computer use

3.8 에이전트 프레임워크·코딩 에이전트

직접 루프 대신 쓸 수 있는 공식 경로는 둘: **Claude Agent SDK**(Messages, `ANTHROPIC_BASE_URL=https://api.meta.ai`, `ANTHROPIC_AUTH_TOKEN`, 모든 모델 alias를 Meta 모델로 고정) · **Codex app-server**(Responses, `wire_api="responses"`, catalog 파일). 터미널 코딩 에이전트는 OpenCode·Codex·Claude Code·Muse Code. 무인 모드(`bypassPermissions`)는 샌드박스 안에서만. — Agent frameworks, Coding agents

3.9 Meta가 권장하는 아키텍처 (요약)

1. Responses API 기본, `previous_response_id` 로 상태 유지
2. Wonder 메인 루프 / Spark 서브에이전트·고볼륨 (한국은 Spark만)
3. 도구 실행·인자 검증은 내 코드
4. 도구 많으면 tool search + namespace
5. 안정 프롬프트 앞, 캐시 모니터링
6. effort는 작업별, 출력 예산 넉넉히
7. 구조화 출력으로 추출·분류·자기평가
8. 종료 조건·타임아웃· `status: incomplete` 처리는 하네스 책임("Treat a run that finishes without a final message as a failure")
9. 무인·안전 확인은 사람의 결정 지점
10. 어댑터별 base URL·인증 정확히

---

4. 에이전트 동작 방식 — 공식 패턴과 예제 코드

4.1 쿡북 agent patterns (인덱스, GitHub)

레시피	내용	URL
Basic agent loop	perceive-decide-act 루프	https://dev.meta.ai/docs/cookbook/basic-agent-loop
Interleaved reasoning and tool use	추론과 도구 호출 교차	https://dev.meta.ai/docs/cookbook/interleaved-reasoning-tool-use
Multi-turn context management	긴 실행의 컨텍스트 관리	https://dev.meta.ai/docs/cookbook/multi-turn-context-management
Search-and-replace edits	안전한 편집	https://dev.meta.ai/docs/cookbook/search-and-replace-edits
Alert fatigue copilot	strict JSON 자기평가로 알림 선별	https://dev.meta.ai/docs/cookbook/alert-fatigue-copilot

4.2 쿡북 use cases (인덱스)

- **Multi-agent orchestration** — 역할 4개(pm/backend/frontend/gtm)가 Kanban 코멘트로만 협상: "every cross-role decision is a comment on a Kanban task, never a back-channel chat, so the run is replayable". 문서 · 코드
- **GitHub agent** — 읽기 전용 → 라벨로 게이트된 수정, "every code change lands as a PR for human review". 문서 · 코드
- **Local agent that asks first** — Glimmer 로컬 → 승인 후에만 Spark로 에스컬레이션. 문서
- 그 외: sandboxed-execution, computer-use, computer-use-macos, browser-verified-web-design, perception-grounding, rayban-openclaw, generating-slides, iterative-game-dev, one-shot-game-dev, perception-chart-analysis, perception-error-screenshot-fix (모두 `https://dev.meta.ai/docs/cookbook/<slug>`)

4.3 Muse Code 하네스 패턴 (쿡북, 문서)

"the harness building blocks for safety, recovery, and orchestration": audit-agent-sessions(이벤트 로그), deterministic-replay(CI 골든 픽스처), staged-approvals, contained-execution, immutable-guardrails, subagent-fanout, goal-tracking("the harness audits the work against acceptance checks before the goal can close"), bundled-skills, loop-and-cron, side-chats. 블로그: "every subagent it spawns, every tool call, every steer and cancel, is observable and replayable through the event log." — Build with Muse Code

- Muse Code는 코딩 작업용. 자체 앱·에이전트는 API 직접 호출이 권장("Call the API directly when you build your own agent or app"). 제품에 넣으려면 `muse exec` (headless, JSONL) 또는 muse-code-sdk.
- Workflows(JS 정의, 최대 1,000 child, 동시 16)는 공개 Apple Silicon 패키지에 아직 미포함. — Workflows, Extending

4.4 "검증 가능한 에이전트" 교재

legal-agent-workshop: Session 1은 기본 루프("ends with an unverified memo"), Session 2는 검증 레이어(ground truth 대조, refusal/escalation, 사람 서명). Agents 102 주제와 같다. Build with Muse Spark 블로그도 "에이전트에 객관적 체크(실패하는 테스트)를 주라"고 권장.

4.5 공개 참가작에서 보이는 패턴 (우승작 아님)

yebo-muse-agent "Muse Spark proposes. Yebo authorizes" + prompt-injection 방어 + 89 tests · musez-home "latency is the product", p50/p95 측정 · sous 카메라 + SAM 3.1 + Spark judge · pocket-muse Baseten `muse-spark-open-1026` 사용.

---

5. 모델 학습(포스트트레이닝)

모델	경로	프레임워크	하드웨어	데이터 포맷	출처
Muse Glimmer 30B	SFT(QLoRA / LoRA / full), RL(GRPO, DPO)	Axolotl 0.19, TRL+PEFT, TitanRL/TorchTitan	QLoRA 1×H100(peak 27GB), 나머지 8×H100. Full은 97GB/GPU + 디스크 222GB	OpenAI `messages` JSONL(이미지 인라인), DPO `prompt/chosen/rejected`, GRPO prompt+reward fn	Glimmer post-training
Muse Spark Open 816B	LoRA/full SFT, DAPO RL, GRPO	TorchTitan(Blackwell 필수), Miles(64×H200), NeMo AutoModel, TRL(64×H100)	수십~수백 GPU, 체크포인트 GPU당 ~1.6TB	JSONL `messages`	Spark Open post-training
호스티드	Together AI(SFT, GRPO — preview), Fireworks(serverless LoRA SFT/DPO/ORPO/distillation/RL, rank ≤64, 동시 8)	—	—	—	같은 페이지, Fireworks training, Together SFT

- **Meta Model API 자체에는 호스티드 파인튜닝이 없다.** 학습된 모델을 Model API에서 서빙하는 경로도 없다. Fireworks는 학습한 LoRA를 전용 배포에서만 서빙("does not serve a private trained adapter on per-token serverless inference").
- 가이드라인: 정답 예시가 있으면 SFT, 자동 검증만 가능하면 RL. 500+ 샘플이면 의미 있는 적응, 5k~10k+면 강한 도메인 성능. 최종 답만으로 SFT하면 chain-of-thought가 억제될 수 있음.
- Glimmer 프롬프팅: `apply_chat_template` 필수, `reasoning_strength` xhigh/high/medium/low, 샘플링 temperature 1.0 / top_p 0.95 / top_k 64, **턴당 도구 호출 1개**. — Glimmer prompting
- 속도: DFlash speculative decoding(draft 1.6GB)으로 긴 추론 가속, MLX 미지원. — spec-decode

Lookout 판단: 8일 안에 학습은 하지 않는다. 대회 후 호주비타몰 예외 처리 로그(원장)가 쌓이면 Glimmer QLoRA(H100 1장, Modal 크레딧) 또는 Fireworks serverless LoRA로 "주문 예외 분류·플레이북 선택" 전용 모델을 만드는 경로가 열려 있다. 원장이 곧 학습 데이터다.

---

6. 한도·비용·사용자별 토큰 제한

6.1 레이트 리밋 (Pricing and rate limits)

모델	RPM	TPM
muse-wonder-1.0	3,000	4,000,000
muse-spark-1.3 / 1.2 / 1.1	3,000 (각)	4,000,000 (각)
muse-spark-*-contributor	150	3,000,000
sam-3.1	3,000	—
muse-image-1.0	150	—
muse-voice-transcribe	—	256 동시 스트림, 16,000/시간

- **스코프는 팀.** 같은 팀의 모든 API 키가 한도를 공유("Keys on the same team share rate limits"). 모델별 독립.
- 초과 시 429 `rate_limit_exceeded` + `Retry-After`. 지수 백오프+지터 0.5-1초 시작, 3-5회. 400/401/402/403/404/413은 재시도 금지. — Error handling
- 모니터링: 응답 헤더 `x-ratelimit-limit-tokens`, `x-ratelimit-remaining-tokens`, `x-ratelimit-limit-requests`, `x-ratelimit-remaining-requests`; `usage` (`cached_tokens`, `reasoning_tokens`). 사전 계산은 `POST /v1/responses/input_tokens`. — Token counting

6.2 지출 한도·사용자별 한도

- **문서에 없다**: spend limit, usage tier, project 스코프, 키별 permission scope, 키별 한도. Discord FAQ는 "Spend Limit 알림이 과금을 자동으로 막는다고 가정하지 말라"고 경고.
- 있는 것: 키 여러 개 발급(앱·개발자당 1키, 독립 revoke), 키는 팀 소속. — Authentication

6.3 앱이 직접 구현해야 하는 사용자별 한도 (Lookout 신뢰 레이어 설계)

1. **서버 프록시**: 클라이언트에 키 노출 금지. 모든 모델 호출은 서버를 거친다.
2. **사용자별 예산 원장**: 호출 전 `input_tokens` 엔드포인트로 사전 카운트 → 응답 `usage` 를 사용자·세션·사건별로 누적 → 공식 단가로 비용 환산 → 임계치(예: 하루 $1, 사건당 $0.30) 초과 시 앱이 거부하고 브리핑에 표시.
3. **사용자별 토큰 버킷**: 팀 TPM 4M을 사용자 수로 나눠 한 사용자의 폭주가 팀 전체 429를 만들지 않게.
4. **선제 스로틀**: `x-ratelimit-remaining-*` 가 낮으면 저우선 작업(Scout 리서치)을 미룬다.
5. **키 분리**: dev/prod·앱별 키. 유출 시 독립 revoke(한도는 여전히 팀 공유).
6. **프롬프트 캐싱으로 비용 절감**: 플레이북·정책 프롬프트를 앞에 고정, `cached_tokens` 확인($1.25 → $0.15).

6.4 Contributor 모드

`muse-spark-1.3-contributor` 는 $0.10/$0.20으로 싸지만 프롬프트·코드·파일·응답을 Meta 학습에 쓰는 데 동의하는 조건. 고객 데이터가 흐르는 Lookout에는 쓰지 않는다. Standard 모드 콘텐츠는 학습에 쓰지 않는다.

---

7. 로컬 실행 (Glimmer)

- 다운로드: `hf download meta-models/Muse-Glimmer-30B-GGUF --include "Muse-Glimmer-30B-KQuant-17GB-Q4_K_M.gguf" --include "mmproj-Muse-Glimmer-30B-Q4_K_M.gguf"` . 키·게이트 없음, 오프라인 동작. — Get the model
- 런타임: Ollama 0.32.7+(`ollama run muse-glimmer:30b-mlx` , OpenAI 호환 `localhost:11434/v1`), llama.cpp b10353+(`llama serve … -c 131072`), SGLang MLX(48GB+ 통합 메모리, 텍스트만), vLLM(NVIDIA), ExecuTorch. — Deploy
- 메모리: Q4_K_M + 비전 프로젝터 + 131K 컨텍스트 = 19.0GiB. — Quantization
- 주의: temperature 0 금지("Don't run this model greedy"), `<|eom|>` 에서 멈추지 말 것(도구 호출 잘림), 턴당 도구 호출 1개, reasoning을 끌 수 없음, 긴 추론은 streaming.
- 호스팅 폴백: Together AI `meta-models/Muse-Glimmer-30B` ($0.35/$1.50; "Launching soon" 배너 — 가용성 확인 필요), Baseten `meta-models/muse-spark-open-1026` (해커톤 ID, Baseten 페이지에는 없고 참가자 가이드 전사본에만).

---

8. 파트너 리소스

파트너	제공	모델 ID	URL
Baseten	Spark Open 추론, $100 + 2M TPM	`meta-models/muse-spark-open` (문서) / `meta-models/muse-spark-open-1026` (해커톤)	https://www.baseten.co/library/muse-spark-open/
Modal	Spark Open 공유 엔드포인트, 서버리스 GPU·샌드박스·학습, $100	`meta-models/Muse-Spark-Open-816B-A42B`	https://modal.com/library/meta/muse-spark-open , https://modal.com/docs/guide/sandboxes
Fireworks	Spark Open·Glimmer 추론, serverless LoRA 학습	`accounts/fireworks/models/muse-spark-open` , `muse-glimmer-30b`	https://fireworks.ai/models/fireworks/muse-spark-open , https://fireworks.ai/models/fireworks/muse-glimmer-30b
Together AI	Glimmer·Spark Open 추론, SFT/GRPO 포스트트레이닝	`meta-models/Muse-Glimmer-30B`	https://www.together.ai/models/muse-glimmer , https://docs.together.ai/docs/post-training/quickstart-sft
Groq / OpenRouter	Spark Open	`meta-models/muse-spark-open` / `meta/muse-spark-open`	https://console.groq.com/docs/model/meta-models/muse-spark-open , https://openrouter.ai/meta/muse-spark-open
Hugging Face	가중치	`meta-models/Muse-Spark-Open-816B-A42B` , `meta-models/Muse-Glimmer-30B`	https://huggingface.co/meta-models

---

9. Lookout 적용 — 조사 결과가 설계에 주는 변경

영역	현재 설계	조사 후 조정
API	OpenAI 클라이언트, 프로토콜 미정	**Responses API 고정.** 사건(Exception) 하나 = `previous_response_id` 체인 하나. 추론 연속성과 구조화 출력·PDF 입력을 한 곳에서
Resolver 출력	Action[] 구조화 출력	`text.format` json_schema, 깊이 10 이내, 재귀 없음. `strict: true` 로 서버 검증까지
도구	어댑터 함수	`tools` function 정의 + 앱 실행 + `function_call_output`. `tool_choice` 는 auto뿐이므로 "반드시 Plan을 내라"는 구조화 출력으로 강제
Find	Glimmer 분류	Ollama MLX, temperature 1.0 유지, 도구 호출 없이 분류만(턴당 1개 제약 회피). 긴 추론은 `reasoning_strength: low`
신뢰 레이어	원장·게이트·멱등·비용 상한	**사용자별 예산 원장과 토큰 버킷 추가**(§6.3). 원장에 `usage` · `cached_tokens` · `reasoning_tokens` 저장. `x-ratelimit-remaining-*` 기록
승인	WhatsApp 버튼·음성	Muse Code "staged approvals" 패턴 용어를 README에서 차용("Spark proposes, the seller authorizes")
오픈 웨이트 보너스	Glimmer 로컬	Baseten `muse-spark-open-1026` 를 Resolver 폴백으로 붙여 "오픈 웨이트만으로도 동작" 경로 확보
컴퓨터 사용(P3)	Spark	Responses 전용, 스크린샷 크기 고정, `pending_safety_checks` 는 승인 큐로
프롬프트	—	플레이북·정책·출력 스키마를 프롬프트 앞에 고정, 사건 데이터는 뒤에 → 캐시 적중
학습	없음	대회 후: 원장 → Glimmer QLoRA(예외 분류·플레이북 선택)
검증	사후 조건 재조회	legal-agent-workshop 식 "검증 레이어"를 영상에서 한 장면으로: 실행 후 Shopify 재조회로 결과 확인

---

10. 링크 전체 목록

해커톤·발표

- https://cerebralvalley.ai/e/meta-ai-developer-hackathon · https://cerebralvalley.ai/e/meta-ai-developer-hackathon/details (로그인) · https://cerebralvalley.ai/e/meta-ai-developer-hackathon/waiver
- https://dev.meta.ai/events/global-hackathon
- https://developers.meta.com/blog/meta-connect-recap/
- https://x.com/MetaforDevs/status/2110045152601633114 · https://x.com/MetaforDevs/status/2112971571321332179
- https://github.com/meta-ai-hackathon/meta-hack (참가자 가이드 전사본, 비공식)
- https://madrobot.blog/2026/10/03/meta-global-ai-developer-hackathon-1-million-prizes-muse-model-api/

Model API 문서

- 시작: https://dev.meta.ai/docs/overview · https://dev.meta.ai/docs/quickstart · https://dev.meta.ai/docs/authentication · https://dev.meta.ai/docs/models · https://dev.meta.ai/docs/sdks · https://dev.meta.ai/docs/pricing-rate-limits
- 기능: https://dev.meta.ai/docs/tool-calling · https://dev.meta.ai/docs/tool-search · https://dev.meta.ai/docs/search-grounding · https://dev.meta.ai/docs/structured-output · https://dev.meta.ai/docs/reasoning · https://dev.meta.ai/docs/prompt-caching · https://dev.meta.ai/docs/token-counting · https://dev.meta.ai/docs/file-handling · https://dev.meta.ai/docs/image-understanding · https://dev.meta.ai/docs/video-understanding · https://dev.meta.ai/docs/image-generation · https://dev.meta.ai/docs/speech-to-text · https://dev.meta.ai/docs/media-segmentation · https://dev.meta.ai/docs/computer-use · https://dev.meta.ai/docs/error-handling
- 프로토콜: https://dev.meta.ai/docs/protocols · https://dev.meta.ai/docs/protocols/responses · https://dev.meta.ai/docs/protocols/chat-completions · https://dev.meta.ai/docs/protocols/messages · https://dev.meta.ai/docs/api-reference
- 에이전트: https://dev.meta.ai/docs/agent-frameworks · https://dev.meta.ai/docs/coding-agents
- 오픈 웨이트: https://dev.meta.ai/docs/open-weight-models · https://dev.meta.ai/docs/muse-glimmer · https://dev.meta.ai/docs/muse-glimmer/get-the-model · https://dev.meta.ai/docs/muse-glimmer/prompting · https://dev.meta.ai/docs/muse-glimmer/quantization · https://dev.meta.ai/docs/muse-glimmer/spec-decode · https://dev.meta.ai/docs/muse-glimmer/deploy · https://dev.meta.ai/docs/muse-glimmer/hardware-platforms · https://dev.meta.ai/docs/muse-glimmer/post-training · https://dev.meta.ai/docs/muse-spark-open · https://dev.meta.ai/docs/muse-spark-open/get-the-model · https://dev.meta.ai/docs/muse-spark-open/prompting · https://dev.meta.ai/docs/muse-spark-open/deploy · https://dev.meta.ai/docs/muse-spark-open/post-training
- SAM: https://dev.meta.ai/docs/sam/overview · https://dev.meta.ai/docs/sam/segmenting · https://dev.meta.ai/docs/sam/reading-segmentation · https://dev.meta.ai/docs/sam/client-libraries
- 정책: https://dev.meta.ai/legal/geographic-use-policy

쿡북·예제

- https://dev.meta.ai/docs/cookbook · https://dev.meta.ai/docs/cookbook/api-fundamentals · https://dev.meta.ai/docs/cookbook/agent-patterns · https://dev.meta.ai/docs/cookbook/use-cases · https://dev.meta.ai/docs/cookbook/muse-code · https://dev.meta.ai/docs/cookbook/run-locally · https://dev.meta.ai/docs/cookbook/post-training · https://dev.meta.ai/docs/cookbook/muse-image · https://dev.meta.ai/docs/cookbook/muse-voice-transcribe · https://dev.meta.ai/docs/cookbook/sam
- 레시피: basic-agent-loop · interleaved-reasoning-tool-use · multi-turn-context-management · search-and-replace-edits · alert-fatigue-copilot · multi-agent-orchestration · github-agent · glimmer-api-agent · sandboxed-execution · computer-use · computer-use-macos · browser-verified-web-design · perception-grounding · audit-agent-sessions · deterministic-replay · staged-approvals · contained-execution · immutable-guardrails · subagent-fanout · goal-tracking · loop-and-cron · quickstart-chat-completions · streaming-responses · tool-function-calling · structured-output · prompt-caching · reasoning-thinking-tokens · vision-input · long-context · error-handling-retry · search-grounding (모두 `https://dev.meta.ai/docs/cookbook/<slug>`)
- GitHub: https://github.com/meta-models · https://github.com/meta-models/meta-model-cookbook · https://github.com/meta-models/meta-oss-cookbook · https://github.com/meta-models/legal-agent-workshop · https://github.com/meta-models/muse-code-sdk · https://github.com/meta-models/meta-sam · https://huggingface.co/meta-models
- 블로그: https://dev.meta.ai/resources/blog/build-with-muse-spark/ · https://dev.meta.ai/resources/blog/build-with-muse-spark-open · https://dev.meta.ai/resources/blog/build-with-muse-code/ · https://dev.meta.ai/resources/blog/build-with-muse-wonder · https://research.meta.ai/blog/muse-spark-open

Muse Code

- https://dev.meta.ai/docs/muse-code · https://dev.meta.ai/docs/muse-code/workflows · https://dev.meta.ai/docs/muse-code/extending · https://dev.meta.ai/docs/muse-code/permissions · https://dev.meta.ai/docs/muse-code/configuration · https://meta-models.github.io/muse-code-sdk

세션·영상

- https://developers.meta.com/resources/videos/building-with-muse-spark-and-muse-code · https://developers.meta.com/resources/videos/multimodal-intelligence-of-muse-spark · https://dev.meta.ai/resources/videos/

파트너

- https://www.baseten.co/library/muse-spark-open/ · https://modal.com/library/meta/muse-spark-open · https://modal.com/blog/meta-muse-spark-open-now-available-on-modal · https://modal.com/docs/guide/sandboxes · https://modal.com/docs/guide/training-quickstart · https://fireworks.ai/models/fireworks/muse-spark-open · https://fireworks.ai/models/fireworks/muse-glimmer-30b · https://docs.fireworks.ai/fine-tuning/training-api/serverless · https://www.together.ai/models/muse-glimmer · https://www.together.ai/models/muse-spark-open · https://docs.together.ai/docs/post-training/quickstart-sft · https://docs.together.ai/docs/post-training/quickstart-rl · https://console.groq.com/docs/model/meta-models/muse-spark-open · https://openrouter.ai/meta/muse-spark-open · https://recipes.vllm.ai/meta-models/Muse-Glimmer-30B · https://docs.sglang.io/cookbook/autoregressive/Meta/MuseSparkOpen · https://unsloth.ai/docs/models/muse-spark-open

과거 해커톤·참가작

- https://ai.meta.com/blog/llamacon-hackathon/ · https://ai.meta.com/blog/llama-3-hackathon-recap-cerebral-valley/ · https://about.fb.com/news/2024/11/metas-llama-impact-hackathon-pioneering-ai-solutions-for-public-good/ · https://metallama3.devpost.com/project-gallery
- https://github.com/TheCl3m/llama-hack · https://github.com/michaelwaves/llamacon-hackathon · https://github.com/torayeff/llamacon-hackathon-2025-sf/ · https://github.com/williamjsdavis/geo-lm · https://github.com/BasedHardware/openglass
- https://github.com/SaintSinC/pocket-muse · https://github.com/Unclefole/yebo-muse-agent · https://github.com/ztdgtll/musez-home · https://github.com/kopias/sous

---

확인하지 못한 것

- 공식 참가자 가이드 원문(로그인 필요)과 라이브 세션 녹화(Discord `#resources`). 심사 가중치(동일 vs 30/25/25/20), Fireworks 크레딧($100 vs $25), Baseten 해커톤 모델 ID는 전사본 출처라 원문 대조 필요.
- 쿡북 개별 레시피 본문(29개)은 인덱스와 GitHub README 수준으로만 확인.
- 가격·한도 수치는 2026-10-28 기준이며 문서 요약을 거친 값이라 발표 자료에 쓰기 전 원문 재확인 권장.
- Muse Wonder 한국 미제공은 models 페이지 문구이고 실제 호출 결과는 1일차에 확인.

부록 A. 에이전트 설계·실행 — 페이지별 상세 노트

조사일: 2026-10-28. 12개 페이지 모두 응답했고 404는 없었다. 단, 모든 페이지에 "Please confirm your identity" SSO 오버레이가 붙어 있어 쿡북 인덱스 3개(agent-patterns, use-cases, api-fundamentals)는 레시피 제목·한 줄 설명·링크만 읽혔다. 개별 레시피 본문은 이번 범위 밖이다.

공통 사실(여러 페이지에서 반복): base URL은 OpenAI SDK용 `https://api.meta.ai/v1`, Anthropic SDK용 `https://api.meta.ai` (SDK가 `/v1/messages` 를 붙임). 모델 ID는 `muse-wonder-1.0` (메인 루프, 복잡한 작업), `muse-spark-1.3` (저렴·빠름, 서브에이전트), `muse-spark-1.3-contributor`. 인증은 bearer token.

---

1. Agent patterns (쿡북 인덱스)

URL: https://dev.meta.ai/docs/cookbook/agent-patterns

요약

- 모델을 에이전트로 바꾸는 "루프" 레시피 모음 인덱스. 계획·자기수정·장기 실행의 일관성을 다룬다.
- 본문에 코드·파라미터는 없고 5개 레시피 제목과 한 줄 설명만 있다.
- perceive-decide-act 루프, reasoning과 tool use의 interleave, 멀티턴 컨텍스트 관리, search-and-replace 편집, 알림 피로 코파일럿(strict JSON로 자기 평가)을 다룬다.

인용

- "Turn a model into an agent."
- "Wire up the core perceive-decide-act loop."

하위 링크

제목	URL
Basic agent loop	https://dev.meta.ai/docs/cookbook/basic-agent-loop
Interleaved reasoning and tool use	https://dev.meta.ai/docs/cookbook/interleaved-reasoning-tool-use
Multi-turn context management	https://dev.meta.ai/docs/cookbook/multi-turn-context-management
Search-and-replace edits	https://dev.meta.ai/docs/cookbook/search-and-replace-edits
Alert fatigue copilot	https://dev.meta.ai/docs/cookbook/alert-fatigue-copilot

---

2. Tool calling

URL: https://dev.meta.ai/docs/tool-calling

요약

- 흐름: 개발자가 함수를 정의 → 모델이 호출 결정 → 앱이 실행 → 결과를 되돌려 모델이 이어감. 모델은 개발자 도구를 직접 실행하지 않는다.
- 도구 타입은 `function` (JSON Schema 파라미터)과 `custom` (자유 텍스트 입력, `parameters` 없음, Responses 전용).
- Responses API(에이전트 권장)와 Chat Completions 양쪽 사용법, 단일턴/멀티턴/병렬 호출, 스트리밍 인자 수신.
- 내장 도구(`web_search`, `tool_search`)는 서버에서 실행된다.

핵심 파라미터·패턴

- `tools` 배열 사용(`functions` / `function_call` 은 deprecated).
- `tool_choice` : `"auto"` 만 지원. `"none"`, `"required"`, 함수 지정은 HTTP 400.
- `parallel_tool_calls` : 기본 `true`. 턴당 한 번만 호출하려면 `false`.
- `max_tool_calls` : 내장 도구만 제한(최소 1). 함수 도구에는 효과 없음.
- `strict` : 기본 `false`. `true` 면 서버가 스키마를 strict 서브셋으로 검증.
- `defer_loading: true` : tool search와 함께 대규모 도구 세트에 사용.
- Responses 출력: `function_call` 아이템(`name`, `arguments`, `call_id`, `status`). Chat Completions: `tool_calls` + `finish_reason: "tool_calls"`.
- 스트리밍 이벤트: `response.function_call_arguments.delta` / `.done`.

Responses API 결과 반환:

```
response = client.responses.create(
    model="muse-spark-1.3",
```

```
    input=[{
        "type": "function_call_output",
        "call_id": item.call_id,
        "output": json.dumps(result),
    }],
    previous_response_id=response.id,
    tools=tools,
)
```

Chat Completions 결과 반환:

```
messages.append({"role": "tool", "tool_call_id": tc.id, "content": json.dumps(result)})
```

제약·주의

- 함수 이름은 `^[a-zA-Z0-9_.-]+$`, 점은 최대 1개(2개 이상은 400). custom 도구는 예외.
- `custom` 도구를 Chat Completions에 보내면 400.
- `strict: true` 라도 생성된 인자가 스키마에 맞는다는 보장은 없다. 코드에서 검증할 것.
- `tool` 메시지 앞에 `tool_calls` 를 가진 assistant 메시지가 빠지면 400.
- `previous_response_id` 없이 보내면 모든 `function_call_output.call_id` 가 같은 요청의 `function_call.call_id` 와 일치해야 하고, 순서도 호출 순서대로.
- `call_id` / `tool_call_id` 는 1-64자.

인용

- "The model does not execute developer-defined tools itself."
- "Always parse and validate arguments in your own code before using them."

---

3. Tool search

URL: https://dev.meta.ai/docs/tool-search

요약

- 수백 개 도구를 매 턴 프롬프트에 싣지 않고 쓰는 기능. 지연(deferred) 도구는 이름·설명만 보이고 파라미터 스키마는 로드 전까지 숨긴다.
- 로드된 도구는 컨텍스트 **끝**에 붙어 앞선 캐시 프리픽스를 보존한다.
- 설정은 두 가지: `tools` 에 `{"type": "tool_search"}` 추가, 지연할 함수에 `defer_loading: true`.
- Hosted 모드: API가 선언된 지연 도구를 검색해 같은 응답에서 로드. 요청 시점에 도구 목록을 다 아는 경우.
- Client-executed 모드(`execution: "client"`): 모델이 검색 요청을 내고 앱이 조회해 로드할 도구를 돌려줌. 테넌트별 통합처럼 API가 모르는 상태에 의존할 때.
- 함수를 `namespace` 로 묶으면 검색 면이 깔끔해지고 토큰을 더 아낀다.
- Responses와 Messages API에서 동작. Chat Completions는 불가.

핵심 패턴

```
tools=[{
    "type": "tool_search",
    "execution": "client",
    "description": "Find the project-specific tools needed to continue the task.",
    "parameters": {"type": "object",
                   "properties": {"goal": {"type": "string"}},
                   "required": ["goal"], "additionalProperties": False},
}]
```

```
{"type": "tool_search_output", "execution": "client",
 "call_id": search_call.call_id, "status": "completed", "tools": loaded_tools}
```

- 출력 아이템 순서: `tool_search_call` (`paths` 포함) → `tool_search_output` → 일반 `function_call`.
- 예제는 `muse-spark-1.3` + `parallel_tool_calls=False`.

제약·주의

- 요청당 `tool_search` 도구는 하나만(초과 시 400). Hosted 모드는 `defer_loading: true` 도구가 최소 1개 필요. 지연 도구가 있으면 `tool_search` 가 반드시 있어야 함.
- `tool_search` 는 예약된 이름.
- `json_schema` 응답 포맷(`text.format`)과 함께 쓰면 400. `reasoning.effort: "none"` 과도 400.

- 네임스페이스당 함수 10개 미만 권장.
- Client 모드에서는 `tool_search_output` 에 넣은 도구만 호출 가능.
- 로드된 도구 세트를 이후 턴에서 바꾸면 그 지점부터 캐시가 깨진다.

인용

- "Build agents with hundreds of tools without loading the whole library on every turn."
- "Changing the loaded set on a later turn breaks the cache from that point forward, so keep it stable once loaded."

---

4. Structured output

URL: https://dev.meta.ai/docs/structured-output

요약

- JSON Schema를 주면 디코딩 자체를 제약해 스키마에 맞는 JSON만 생성(사후 검증이 아님).
- 용도: 추출, 분류, 함수 인자 생성, 설정 생성.
- OpenAI SDK의 Pydantic 헬퍼(`client.beta.chat.completions.parse`)로 타입 객체를 받을 수 있다.
- 스키마 크기·구조 제한과 `strict` 플래그 설명.

핵심 패턴

```
response_format={
    "type": "json_schema",
    "json_schema": {"name": "Address",
                    "schema": {"type": "object", "properties": {...}, "required": [...]}},
}
```

```
response = client.beta.chat.completions.parse(
    model="muse-spark-1.3", messages=[...], response_format=Address)
address = response.choices[0].message.parsed
```

- Chat Completions는 `response_format` , Responses API는 `text.format` . 서로 바꿔 쓰면 받아들여져도 구조화 출력이 설정되지 않는다.
- 결과는 `choices[0].message.content` 에 JSON 문자열.

제약·주의

- 재귀 스키마 미지원(400). 고정 깊이로 평탄화할 것.
- 중첩 깊이 10, 속성 총 5,000개, 이름·정의·enum·const 문자열 합 120,000자, enum 1,000개(앱별 상향 가능), `$ref` 인라인 후 200,000 노드 초과 거부.
- 같은 제한이 `/v1/chat/completions` 와 `/v1/responses` 의 함수 도구 `parameters` 에도 적용.
- `strict: true` : 루트는 plain object, `allOf` / `oneOf` 불가, 모든 object에 `additionalProperties: false` , `required` 에 모든 속성 나열.
- 출력 측면에서 `strict: false` 는 현재 `strict: true` 와 같게 동작(항상 제약 디코딩). `strict` 는 서버 스키마 검증 여부만 바꾼다.
- 어떤 모델이 지원하는지는 페이지에 명시 없음(예제는 `muse-spark-1.3`).

인용

- "The model constrains token generation to produce only valid JSON matching your schema."
- "A request containing a recursive JSON schema returns `HTTP 400` ."

---

5. Reasoning

URL: https://dev.meta.ai/docs/reasoning

요약

- Muse Wonder·Muse Spark는 답하기 전에 내부 추론을 하며 `reasoning_effort` 로 양을 조절.
- 추론 토큰은 출력 토큰 한도에 포함되고 출력 토큰으로 과금.
- 원문 chain of thought는 비공개. Responses API에서는 요약(summary)만 요청 가능.
- 멀티턴 추론 연속성은 Responses API에서만. Chat Completions는 외부 키에 대해 `reasoning_content` 가 빈 값으로 redact되어 턴 간 추론이 이어지지 않음.
- 다단계 문제는 effort를 높이고, 직답형 작업은 낮춘다.

핵심 파라미터

- effort 값: `"none"` (Wonder/Spark에서 400), `"minimal"` , `"low"` , `"medium"` , `"high"` , `"xhigh"` , `"max"` (`muse-wonder-1.0` , `muse-spark-1.3` 만; `-contributor` 는 불가). 생략하면 모델이 선택.

- Chat Completions: 최상위 `reasoning_effort="high"`. Responses: `reasoning={"effort": "high"}`.
- 멀티턴(Responses): `previous_response_id` 로 체인하거나, stateless replay는 `include: ["reasoning.encrypted_content"]` 를 요청해 `reasoning` 아이템을 그대로 되돌려 보냄.
- 요약: `"reasoning": {"effort": "high", "summary": "auto"}` (`auto` / `concise` / `detailed`). 스트리밍은 `response.reasoning_summary_text.delta` / `.done`.
- 사용량: `completion_tokens_details.reasoning_tokens` (CC), `output_tokens_details.reasoning_tokens` (Responses).
- 내장 도구 루프(`web_search` 등) 중 reasoning 아이템은 `content` 가 비어 있지만 `encrypted_content` 는 유지되어 stateless replay가 된다.

제약·주의

- `max_tokens` / `max_output_tokens` 가 추론+가시 출력 모두를 덮으므로 넉넉히 잡을 것. `"max"` 에서는 추론 비중이 커진다.
- logprobs 미지원(400).
- `"max"` 를 `muse-wonder-1.0` 에 줘도 Muse Wonder Full이 선택되지는 않음(Muse Code 전용).
- `generate_summary` 는 deprecated. `summary` 배열이 `[]` 일 수 있음.
- 스트리밍 첫 청크까지 초기 지연이 있다.

인용

- "Reasoning tokens count toward your output-token budget"
- "Chat Completions cannot carry reasoning across turns for external API keys."

---

6. Prompt caching

URL: https://dev.meta.ai/docs/prompt-caching

요약

- 자동 prefix caching. 플래그·키·breakpoint 불필요.
- 프롬프트 시작부터 매칭하므로 안정적인 내용(system prompt, 지시, 예시)을 앞에, 변하는 내용(사용자 메시지)을 뒤에 둔다.
- 캐시 적중은 `usage` 에 보고됨: Responses `usage.input_tokens_details.cached_tokens`, CC `usage.prompt_tokens_details.cached_tokens`.
- 캐시 토큰은 입력 토큰의 부분집합이며 할인 과금(비율은 페이지에 없음, Pricing 참조).
- `prompt_cache_key` (deprecated `user` 대체)는 고볼륨에서 적중률을 높이는 라우팅 힌트.
- Responses 전용 `prompt_cache_retention`: `"in_memory"` (기본) 또는 `"24h"`.
- `store` 파라미터와 무관하게 캐싱된다.

핵심 패턴

```
response = client.chat.completions.create(
    model="muse-spark-1.3",
    prompt_cache_key="my-app-system-prompt",
    messages=[...])
```

```
{"model": "muse-spark-1.3", "prompt_cache_retention": "24h",
 "input": "Summarize the incident report from this quarter."}
```

제약·주의

- 첫 번째로 다른 토큰에서 매칭이 끝난다. 타임스탬프를 앞에 두면 재사용 프리픽스가 줄어든다.
- 새 프리픽스 첫 요청이나 eviction 후 0은 정상. retention은 힌트일 뿐 보장 아님.
- 사용자별·세션별 키는 과분할로 적중률을 낮춘다.
- `previous_response_id` 사용 시 후속 턴은 새 턴 토큰만 보고하므로 전체 프리픽스 계정을 보려면 `input` 배열을 직접 관리.
- `store: false` 는 `GET /v1/responses/{id}` 404를 만들지만 캐싱은 막지 않는다.
- 기본 TTL, 최소 캐시 길이, 정확한 할인율은 페이지에 없음.

인용

- "caches the stable prefix automatically — no flag or key to manage"
- "prompt_cache_retention is a hint, not a guarantee."

---

7. Agent frameworks

URL: https://dev.meta.ai/docs/agent-frameworks

요약

- 완성된 터미널 에이전트 대신 직접 에이전트 루프를 짜는 법. 프롬프트·도구·제어 흐름은 내 코드가 정의.
- 다루는 프레임워크는 **두 개뿐**: Claude Agent SDK(Messages API), OpenAI Codex app-server(Responses API). LangChain·OpenAI Agents SDK·Vercel AI SDK는 없다.
- 모델 분담: `muse-wonder-1.0` 메인 루프, `muse-spark-1.3` 서브에이전트 등 저렴한 작업.
- Production 패턴: terminal-event 완료 판정, wall-clock/idle 타임아웃, 무인 에이전트 샌드박싱, 도구 권한 범위 제한, 비용 측정, 재시도.

핵심 설정 Claude Agent SDK:

```
ANTHROPIC_BASE_URL=https://api.meta.ai        # /v1 없음; SDK가 /v1/messages 붙임
ANTHROPIC_AUTH_TOKEN=<MODEL_API_KEY>          # Authorization: Bearer
ANTHROPIC_MODEL=muse-wonder-1.0
ANTHROPIC_DEFAULT_OPUS_MODEL=muse-wonder-1.0
ANTHROPIC_DEFAULT_SONNET_MODEL=muse-spark-1.3
ANTHROPIC_DEFAULT_HAIKU_MODEL=muse-spark-1.3
CLAUDE_CODE_SUBAGENT_MODEL=muse-spark-1.3
```

```
const run = query({
  prompt: "Add input validation to the signup handler and run the tests.",
  options: { model: "muse-wonder-1.0", allowedTools: ["Read", "Edit", "Bash"],
    permissionMode: "bypassPermissions", settingSources: [],
    env: { ...process.env, ANTHROPIC_BASE_URL: "https://api.meta.ai", /* ... */ } },
});
```

Codex app-server:

```
-c model="muse-wonder-1.0"
-c model_provider="meta"
-c model_providers.meta.base_url="https://api.meta.ai/v1"
-c model_providers.meta.env_key="MODEL_API_KEY"
-c model_providers.meta.wire_api="responses"
-c model_providers.meta.requires_openai_auth=false
-c model_catalog_json="<path to catalog JSON file>"
```

제약·주의

- `ANTHROPIC_AUTH_TOKEN` 을 쓸 것. `ANTHROPIC_API_KEY` 는 `x-api-key` 헤더를 보내 인증되지 않는다.
- 모든 모델 변수를 Model API 모델로 고정하지 않으면 서브에이전트/haiku 티어가 미제공 Claude 모델을 찾는다.
- Codex의 `/v1/models` refresh는 실패(`missing field 'models'`)하므로 catalog 파일로 모델 등록. `model_catalog_json` 은 파일 경로.
- `bypassPermissions` / `approvalPolicy: "never"` 는 샌드박스 안에서만.
- TLS 변수(`SSL_CERT_FILE` , `NODE_EXTRA_CA_CERTS`)를 env allowlist에 유지.
- Codex는 `turn/completed` 를 내 turn id와 매칭하고, assistant 텍스트 없는 완료는 실패로 처리.
- 끊긴 스트림·502는 backoff 재시도.

인용

- "Treat a run that finishes without a final message as a failure rather than returning an empty result."
- "Use this rather than `ANTHROPIC_API_KEY` , which sends an `x-api-key` header instead."

---

8. Coding agents

URL: https://dev.meta.ai/docs/coding-agents

요약

- 터미널 코딩 에이전트를 Model API로 구동: OpenAI 호환(Responses/CC, `https://api.meta.ai/v1`) 또는 Anthropic Messages(`https://api.meta.ai`).
- 다루는 에이전트: **OpenCode, Codex, Claude Code** (+ Meta 자체 Muse Code). Cursor·Cline은 없음.
- 에이전트가 오케스트레이터, 모델은 완료 텍스트와 tool-call 요청을 돌려준다.
- 비디오 입력은 Responses 전용. 추론 연속성은 surface·어댑터에 따라 다름.
- API 키 관리, 비용 추적, 트러블슈팅 포함.

핵심 설정 OpenCode `opencode.json` (Responses 어댑터):

```
{"provider": {"meta": {"name": "Meta Model API", "npm": "@ai-sdk/openai",
  "options": {"baseURL": "https://api.meta.ai/v1"},
```

```
  "models": {"muse-spark-1.3": {"name": "muse-spark-1.3", "reasoning": true,
    "options": {"reasoningEffort": "high", "reasoningSummary": "auto",
      "include": ["reasoning.encrypted_content"]}}}}}
```

Codex `~/.codex/config.toml`:

```
model = "muse-wonder-1.0"
model_provider = "meta"
model_reasoning_effort = "high"
model_context_window = 1048576
[model_providers.meta]
name = "Meta Model API"
base_url = "https://api.meta.ai/v1"
env_key = "MODEL_API_KEY"
wire_api = "responses"
```

Claude Code: agent-frameworks의 env와 동일 + `export ENABLE_TOOL_SEARCH="true"`.

제약·주의

- `@ai-sdk/openai-compatible` (Chat Completions) 어댑터는 턴 간 추론을 잃어 반복 작업·컨텍스트 유실이 생길 수 있다. PDF도 이 어댑터로는 첨부 안 됨.
- modality 선언이 빠지면 이미지·파일 파트가 조용히 버려질 수 있다.
- base URL에 trailing slash 금지. Claude Code는 `/v1` 없이.
- `ANTHROPIC_SMALL_FAST_MODEL` 은 deprecated(구버전은 아직 읽음).
- Codex는 `temperature` / `top_p` 를 보내지 않아 API 기본값 적용. OpenAI에서 로그아웃해야 `meta` provider 사용.
- OpenCode는 provider 설정 변경 후 완전 재시작 필요. 커스텀 provider는 가격 메타데이터가 없어 비용이 $0로 보일 수 있음.
- 키는 `/connect` 나 env var로; 커밋되는 설정 파일에 넣지 말 것.

인용

- "Each turn reasons from scratch, so in multi-step and agentic loops the model can lose the thread"
- "The Messages adapter runs stateless (no server-managed conversation state)"

---

9. Computer use

URL: https://dev.meta.ai/docs/computer-use

요약

- `muse-wonder-1.0` + Responses API(computer tool의 유일한 지원 엔드포인트)로 CUA를 만든다.
- 모델이 스크린샷을 보고 마우스·키보드 액션을 구조화해 돌려주면 앱이 실행하고 새 스크린샷을 보내는 observe-act 루프.
- computer tool은 액션 **프로토콜**만 정의. 브라우저/데스크톱, 드라이버(Playwright/Puppeteer/pyautogui), 스크린샷 캡처, 안전 처리는 내가 제공. Meta가 호스팅하는 환경은 없다.
- 상태는 `previous_response_id` (서버) 또는 히스토리 replay(stateless).
- 배칭·대기·완료·불가능 선언을 다루는 엄격한 system prompt 권장.
- 대안: 모델이 쓴 Playwright/pyautogui 코드를 실행하는 함수 도구(code execution).

핵심 패턴

```
{"tools": [{"type": "computer"}]}
```

- 액션: `click`, `double_click`, `drag`, `keypress`, `move`, `scroll`, `type`, `screenshot`, `wait`. 좌표는 마지막으로 보낸 스크린샷의 픽셀, (0,0)은 좌상단.

```
{"type": "computer_call_output", "call_id": "call_abc123",
 "acknowledged_safety_checks": [],
 "output": {"type": "computer_screenshot", "image_url": "<...>"}}
```

```
for _ in range(max_steps):
    calls = [i for i in response.output if i.type == "computer_call"]
    if not calls:
        return response.output_text
    driver.execute(calls[0].actions)
    png = driver.screenshot()
```

```
    response = client.responses.create(model="muse-wonder-1.0",
        previous_response_id=response.id, ...)
```

- `pending_safety_checks` 항목은 `{id, code, message}` ; 수락한 것만 `acknowledged_safety_checks` 에 echo.
- 기타: `store` (기본 true), `truncation` (기본 `"disabled"` , `"auto"` 가능), `instructions` , `max_output_tokens` .

제약·주의

- 스크린샷 크기를 trajectory 내내 고정하고, 좌표를 만든 viewport를 그대로 캡처.
- 디코드된 이미지 25 MiB 이하, 선언한 MIME이 바이트와 일치해야(아니면 400).
- 좌표 액션은 매 턴 스크린샷 필요(`screenshot` / `wait` 만 있는 호출은 예외).
- 액션 실패·예상 밖 네비게이션·모달 발생 시 배치를 중단.
- Stateless replay는 최대 29 call/output 쌍, `meta_safety_replay_receipt` 는 변경 없이 복사.
- `truncation: "auto"` 는 계정에 따라 미활성(400).
- 컨텍스트 1,048,576 토큰. 긴 실행에서는 스크린샷이 토큰을 지배.
- `max_output_tokens` 도달 시 HTTP 200 + `status: "incomplete"` . 예산을 올려 같은 상태에서 계속.
- 권한·자격 증명 부재나 깨진 UI에서는 깔끔히 멈추도록 프롬프트해 무한 루프 방지.

인용

- "The computer tool defines the action **protocol**, not an environment."
- "Treat a safety check as a decision point in your harness, not a field to auto-fill."

---

10. Protocols (Choosing an API)

URL: https://dev.meta.ai/docs/protocols

요약

- 세 가지 요청 포맷: Responses, Chat Completions, Messages. 모두 같은 모델, 같은 bearer 인증, 토큰당 같은 가격.
- 기존 코드가 쓰는 포맷을 고르되, 새 작업의 기본 권장은 Responses.
- 차이는 주로 턴 간 추론 연속성, search grounding, tool search 지원 여부.
- Messages는 stateless라 클라이언트가 히스토리를 재전송.

비교(페이지 "At a glance" 재구성)

포맷	엔드포인트	적합	턴 간 추론	비고
Responses	`POST /v1/responses`	에이전트, 멀티스텝 도구 루프, 전체 기능	예 (encrypted replay 또는 `previous_response_id`)	search grounding, tool search, background 실행, 파일 입력 지원
Chat Completions	`POST /v1/chat/completions`	기존 OpenAI messages 배열 코드	아니오(외부 키)	`function` 도구만. search grounding·tool search 불가
Messages	`POST /v1/messages`	Claude Code 등 Anthropic 포맷 도구	예 (`thinking` replay)	Anthropic SDK를 `https://api.meta.ai` 에, stateless

제약·주의

- base URL이 SDK별로 다르다: OpenAI SDK `https://api.meta.ai/v1` , Anthropic SDK `https://api.meta.ai` .
- 가격이 같으므로 비용은 선택 기준이 아니다.
- CC로 시작했다면 추론 replay나 search grounding이 필요해질 때 Responses로 옮기는 경로를 권장.

인용

- "The recommended default for new work."
- "All three run the same models, use the same bearer-token auth, and cost the same per token."

---

11. Use cases (쿡북 인덱스)

URL: https://dev.meta.ai/docs/cookbook/use-cases

요약

- 멀티모달 인식·오케스트레이션·완성 앱을 결합한 end-to-end 예제 인덱스. 코드·파라미터 없음.
- 범위: 차트/오류 스크린샷 인식, 브라우저로 검증하는 코딩 에이전트, 게임·슬라이드 생성, 샌드박스 실행, 멀티에이전트, 컴퓨터 사용, 스마트 글래스, GitHub Actions 봇, 로컬 우선 에이전트.

- "Local agent that asks first"는 Muse Glimmer를 로컬에서 돌리고 승인 후에만 Muse Spark로 에스컬레이션.
- 여러 레시피가 실제 브라우저/데스크톱을 요구하며 설정 요건은 인덱스에 없다.

인용

- "Build end-to-end: multimodal perception, orchestration, and complete apps you can adapt."
- "Build a complete 3D browser game in one pass — one structured prompt, no iteration loop."

하위 링크

제목	URL
Chart analysis	https://dev.meta.ai/docs/cookbook/perception-chart-analysis
Error screenshot fix	https://dev.meta.ai/docs/cookbook/perception-error-screenshot-fix
Smart glasses with OpenClaw	https://dev.meta.ai/docs/cookbook/rayban-openclaw
Generating slides	https://dev.meta.ai/docs/cookbook/generating-slides
Browser-verified web design	https://dev.meta.ai/docs/cookbook/browser-verified-web-design
Iterative game dev	https://dev.meta.ai/docs/cookbook/iterative-game-dev
Sandboxed execution	https://dev.meta.ai/docs/cookbook/sandboxed-execution
Multi-agent orchestration	https://dev.meta.ai/docs/cookbook/multi-agent-orchestration
One-shot game dev	https://dev.meta.ai/docs/cookbook/one-shot-game-dev
Perception grounding	https://dev.meta.ai/docs/cookbook/perception-grounding
GitHub agent	https://dev.meta.ai/docs/cookbook/github-agent
Computer use	https://dev.meta.ai/docs/cookbook/computer-use
Computer use on macOS	https://dev.meta.ai/docs/cookbook/computer-use-macos
Local agent that asks first	https://dev.meta.ai/docs/cookbook/glimmer-api-agent

---

12. API fundamentals (쿡북 인덱스)

URL: https://dev.meta.ai/docs/cookbook/api-fundamentals

요약

- 10개 기초 레시피 인덱스. "building block을 하나씩 검증"하는 스타터.
- 범위: chat completions, 스트리밍, tool calling, structured output, prompt caching, reasoning 토큰, vision, long context, 오류 처리·재시도, search grounding.
- 본문에 코드·base URL·모델명은 없다(설명에 "Muse Spark", "OpenAI SDK" 언급만).

인용

- "Validate one building block at a time and walk away with a starter you can extend."
- "Point the OpenAI SDK at a new base URL and make your first Muse Spark call."

하위 링크

제목	URL
Quickstart: chat completions	https://dev.meta.ai/docs/cookbook/quickstart-chat-completions
Streaming responses	https://dev.meta.ai/docs/cookbook/streaming-responses
Tool and function calling	https://dev.meta.ai/docs/cookbook/tool-function-calling
Structured output	https://dev.meta.ai/docs/cookbook/structured-output
Prompt caching	https://dev.meta.ai/docs/cookbook/prompt-caching
Reasoning and thinking tokens	https://dev.meta.ai/docs/cookbook/reasoning-thinking-tokens
Vision input	https://dev.meta.ai/docs/cookbook/vision-input
Long context	https://dev.meta.ai/docs/cookbook/long-context

제목	URL
Error handling and retry	https://dev.meta.ai/docs/cookbook/error-handling-retry
Search grounding	https://dev.meta.ai/docs/cookbook/search-grounding

기타 쿡북 섹션(사이드바에서 확인): All recipes `/docs/cookbook`, Building with Muse Code `/docs/cookbook/muse-code`, Muse Image `/docs/cookbook/muse-image`, Muse Voice Transcribe `/docs/cookbook/muse-voice-transcribe`, Segment Anything Model `/docs/cookbook/sam`, Run locally `/docs/cookbook/run-locally`, Post-training `/docs/cookbook/post-training`. 관련 가이드: Search grounding `/docs/search-grounding`, File handling `/docs/file-handling`, Token counting `/docs/token-counting`, Error handling `/docs/error-handling`, Models `/docs/models`, Pricing and rate limits `/docs/pricing-rate-limits`.

Meta가 권장하는 에이전트 아키텍처 (페이지 내용에만 근거)

- **Responses API를 기본으로.** 새 작업의 권장 기본값이고, 에이전트·멀티스텝 도구 루프에 필요한 턴 간 추론 연속성(`previous_response_id` 또는 `reasoning.encrypted_content` replay), tool search, search grounding, 파일·비디오 입력이 여기서만 다 된다. Chat Completions는 턴마다 추론을 처음부터 하므로 에이전트 루프에서 맥락을 잃을 수 있다.
- **모델 분담: Wonder가 메인 루프, Spark가 서브에이전트.** `muse-wonder-1.0` 을 복잡한 작업·메인 루프에, `muse-spark-1.3` 을 서브에이전트와 저렴한 작업에 쓰라고 agent-frameworks·coding-agents 모두 안내한다. Computer use는 Wonder 기준.
- **도구 실행은 전적으로 내 코드.** 모델은 `function_call` 만 내고, 앱이 실행해 `function_call_output` 을 되돌린다. 인자는 `strict` 와 무관하게 코드에서 검증. `tool_choice` 는 `auto` 만 되므로 "반드시 도구 호출" 강제는 프롬프트나 루프 설계로.
- **도구가 많으면 tool search + namespace.** 이름·설명만 노출하고 스키마는 지연 로드, 네임스페이스당 10개 미만, 한번 로드한 세트는 바꾸지 않아 캐시를 지킨다. 구조화 출력(`json_schema`)과는 동시 사용 불가.
- **프롬프트는 안정 부분을 앞에.** 자동 prefix caching이므로 system prompt·지시·예시를 앞에, 사용자 입력을 뒤에 두고 `cached_tokens` 를 모니터링. 고볼륨은 `prompt_cache_key`, 버스트 트래픽은 `prompt_cache_retention: "24h"`.
- **Reasoning effort는 작업에 맞춰 조절하고 출력 예산을 넉넉히.** 추론 토큰이 `max_output_tokens` 에 포함되므로 높은 effort에서 가시 출력이 잘릴 수 있다. `"none"` 은 Wonder/Spark에서 400.
- **구조화 출력으로 추출·분류·자기평가.** 디코딩 자체를 제약하므로 JSON 파싱 실패를 걱정할 필요가 없고, 재귀 스키마·깊이 10 같은 제한만 지키면 된다. agent-patterns의 "Alert fatigue copilot"이 strict JSON 자기평가 예.
- **종료 조건과 타임아웃을 명시적으로.** 최종 메시지 없이 끝난 실행은 실패로 취급하고, wall-clock/idle 타임아웃, 끊긴 스트림·502 재시도(backoff), `status: "incomplete"` 처리까지 하네스가 책임진다.
- **무인 모드는 샌드박스 안에서만.** `bypassPermissions` / `approvalPolicy: "never"`, 그리고 computer use의 safety check 수락은 사람이 판단할 결정 지점으로 다룬다. 로컬 우선(Glimmer) → 승인 후 Spark 에스컬레이션 레시피도 같은 맥락.
- **인증·base URL을 어댑터별로 정확히.** OpenAI SDK는 `https://api.meta.ai/v1`, Anthropic SDK/Claude Code는 `https://api.meta.ai` + `ANTHROPIC_AUTH_TOKEN` (bearer). Claude Code 계열은 모든 모델 alias를 Model API 모델로 고정.

건너뛴 것: 쿡북 개별 레시피 본문(29개 하위 링크)과 `/docs/protocols/responses` 같은 2차 페이지는 열지 않았다. 리스크: 세 쿡북 인덱스는 SSO 오버레이 뒤라 설명이 잘렸을 수 있고, 모델 지원 매트릭스(어느 모델이 structured output을 지원하는지 등)는 페이지에 명시가 없어 공란으로 두었다.

부록 B. 학습·로컬 실행·한도 — 페이지별 상세 노트

조사일 2026-10-28. 13개 페이지 모두 정상 로드(404 없음). 일부 페이지는 "Please confirm your identity" 오버레이가 섞여 있어 본문이 얇게 잡힘 — 해당 페이지는 명시함.

---

1. https://dev.meta.ai/docs/cookbook/post-training

요약

- 포스트트레이닝 쿡북 인덱스 페이지. 본문은 두 레시피 카드와 네비게이션뿐.
- 두 레시피 모두 TorchTitan 기반.
- RL 레시피: GRPO + DAPO loss, DAPO-Math에서 시작해 사용자가 task를 꽂는 구조.
- SFT 레시피: text/image/audio 대화에 대한 full-parameter SFT.
- 명령어·데이터 포맷·하드웨어는 이 페이지에 없음 → 모델별 Post-training 페이지로 이동.

한계/주의: 인덱스 역할만 함. 상세는 2·3번 페이지.

인용

- "RL with TorchTitan GRPO with the DAPO loss on a task you plug in, starting from DAPO-Math."
- "Full-parameter SFT on your own text, image, and audio conversations."

---

2. https://dev.meta.ai/docs/muse-glimmer/post-training

요약

- 대상: `meta-models/Muse-Glimmer-30B`, 자기 GPU에서 오픈소스 프레임워크로 수행.
- 선택 기준: 정답 응답 예시가 있으면 SFT, 자동 검증만 가능하면 RL. 대부분 SFT로 시작, RL은 SFT 체크포인트 위에서.
- 프레임워크 4종: Axolotl 0.19.0 (QLoRA / bf16 LoRA FSDP2 / full bf16 FSDP2), TRL+PEFT (LoRA/QLoRA), TitanRL/TorchTitan (GRPO, verifiable reward), TRL (DPO/GRPO).
- 검증 하드웨어: Axolotl 전 방식 8×H100, QLoRA는 1×H100도 가능. TitanRL 8×H100 95GB (trainer 6 GPU + generator 2 GPU).
- 컨텍스트 128K (131,072). 데이터셋 500+개면 의미 있는 적응, 5k-10k+면 강한 도메인 성능.
- TitanRL 결과: mean reward 0.21→0.98 (15 step), 정답률 52%→100%.

핵심 파라미터/명령

```
# env: Python 3.12, torch==2.12.1, axolotl==0.19.0, flash-attn==2.8.3
python scripts/prepare_dataset.py cosyn-circuit --tokenizer-model meta-models/Muse-Glimmer-30B --max-tokens 4096
axolotl train configs/muse-glimmer-30b-qlora.yaml --max-steps 2 ... --launcher torchrun -- --nproc_per_node=8 --
nnodes=1
python -m torchtitan.rl.train --module torchtitan.rl.examples.glimmer_data_analyst --config
rl_grpo_muse_glimmer_30b_data_analyst_smoke
```

- 데이터 포맷: Axolotl = OpenAI-style `messages` (이미지는 content 안에 typed parts); TRL SFT = JSONL 한 줄당 대화, reasoning 유지하려면 `reasoning_content` 포함; DPO = `prompt` / `chosen` / `rejected`; GRPO = prompt만 + reward 함수(스칼라).
- LoRA 기본: rank 16, alpha 32, `q/k/v/o_proj`, lr 2e-4, 3 epoch. QLoRA: NF4 double-quant, batch 1, grad accum 16. DPO: `beta=0.1`, lr ~5e-7. GRPO: lr 1e-6-3e-6, group 4-16. Full FT: lr ~1e-5.

한계/주의

- Full-param 메모리(30B bf16, N-way shard): weights ~60GB/N + grads ~60GB/N + fp32 Adam ~240GB/N, activation 제외. Adam 상태는 step 2에 잡히므로 step 2 기준으로 사이징.
- Full FSDP2 최종 저장 시 peak 97,340 MiB/GPU — H100 거의 꽉 참. 디스크 ~222GB.
- 이미지는 `messages[*].content` 인라인 필수 (top-level `images` 컬럼은 첫 장만 남음). `sample_packing: false` 필수.
- `deepspeed` extra 설치 금지, `adamw_torch_fused` 사용, bf16 LoRA는 `activation_checkpointing: false`.
- QLoRA 재로드 시 `dtype=torch.bfloat16` + `bnb_4rc_quant_storage=torch.bfloat16` 둘 다 필요.
- TitanRL 설치 시 `torchcomms` 제외. Glimmer KV head 2개라 generator TP 최대 2.
- 최종 답만으로 SFT하면 chain-of-thought 억제될 수 있음. RL은 KL penalty 유지 + held-out eval.

인용

- "Start with SFT when you have target responses."
- "Adam allocates its state on the first step, so memory jumps between step 1 and step 2"

3. https://dev.meta.ai/docs/muse-spark-open/post-training

요약

- 대상: Muse Spark Open, 816B MoE / 42B active. 오픈소스 프레임워크(자기 GPU)와 **호스티드 서비스** 둘 다 소개.
- TorchTitan: LoRA SFT 또는 full SFT(text/image/audio), RL은 TitanRL DAPO + vLLM rollout. 레시피는 NVIDIA Blackwell(SM100+) 필수. LoRA SFT 레시피 `muse_spark_816b_a42b_ultradata_lora_sft_cp16_128k` : rank 8, 131,072 ctx, GB300 16장. DAPO RL 레시피: trainer 256 GPU + generator 16 GPU.
- Miles (GRPO, SGLang rollout): 8노드×8 H200 (64 GPU, InfiniBand), `MILES_SCRIPT_EXTERNAL_RAY=1` . 모드: math (DAPO-Math-17k) / `--vlm` (Geometry3K, vision tower frozen). 기본 32 prompt×16 sample, 4096 응답, 8192 ctx, lr 1e-6.
- NVIDIA NeMo AutoModel (full SFT/LoRA, HF 체크포인트 직접 로드), Megatron Bridge (HF 변환 필요).
- TRL `SFTTrainer` + LoRA: 8노드×8 H100, 500 step, step당 ~14.2s, 40.2GB/GPU, adapter 194MB.
- 호스티드: **Together AI** (SFT, GRPO RL — preview), **Fireworks AI** (serverless LoRA: SFT/DPO/ORPO/distillation/RL; full-param은 전용 GPU). Fireworks base `accounts/fireworks/models/muse-spark-open-nvfp4` .

핵심 파라미터/명령

```
pip install trl peft "transformers>=5.19"       # TRL
pip install "fireworks-ai[training]"            # Fireworks, FIREWORKS_API_KEY
# TRL: fsdp_size=64, ep_size=8, max_length=2048, packing=True, save_only_model=True, r=16, lora_alpha=32
# Miles: scripts/run_muse_spark_64xh200.py, --save-interval, --run-id, tools/convert_muse_spark_torch_dist_to_hf.py
```

- 데이터: TRL = JSONL + chat 형식 `messages` 배열.

한계/주의

- 스토리지: Miles 기준 GPU당 ~1.6TB(BF16 ckpt + Megatron copy), `/scratch` NVMe ~1.2TB.
- Fireworks serverless: LoRA rank ≤64, 시퀀스 ≤262,144 (모델 native 1,048,576), 동시 run 기본 8개. 학습 LoRA는 전용 deployment에서만 서빙(per-token serverless 불가). tokenizer는 NVFP4 base와 일치해야 함. MoE router replay 켜둘 것.
- Fireworks 이름 제약: sampler ckpt ≤17자, `output_model_id` 1-63자 소문자/숫자/하이픈. 세션 삭제 전 체크포인트 promote 필수. 모든 remote op에 `.result()` 호출.
- TRL: full optimizer-state ckpt 미지원 → `save_only_model=True` 필수.
- Miles: `\boxed{}` 최종 답 일치해야 reward 1. `--vlm` 은 dynamic sampling 때문에 rollout당 시간 급증.

인용

- "Muse Spark Open support on Together Post-Training is in preview, so the service and its limits can change."
- "Fireworks does not serve a private trained adapter on per-token serverless inference."

---

4. https://dev.meta.ai/docs/cookbook/run-locally

요약

- 로컬 서빙 쿡북 인덱스. 본문은 소개 한 줄 + Mac 가이드 카드 하나.
- 유일한 구체 항목: "Muse Spark Open을 Mac Studio 2대 또는 4대로 실행" 가이드.
- vLLM/llama.cpp/Ollama/양자화/하드웨어 수치 없음 → 사이드바의 모델별 "Get the model / Recommended settings / Run inference / Hardware platforms / Quantization / Speculative decoding" 페이지로 분산.
- identity 오버레이가 있어 일부 본문이 가려졌을 가능성.

한계/주의: 이 페이지만으로는 로컬 실행 절차를 알 수 없음. Glimmer 로컬 실행 실무 정보는 13번(Get the model)과 6번(spec-decode)에 있음.

인용

- "Serve a Muse open-weight model on your own machines, step by step."
- "Run Muse Spark Open on two or four Mac Studios."

---

5. https://dev.meta.ai/docs/muse-glimmer/prompting

요약

- 프롬프트는 반드시 tokenizer의 `apply_chat_template` 로 생성, 추론 시 `add_generation_prompt=True` .
- 턴 구조: `<|start|>{role}<|message|>…<|eot|>` (턴이 이어지면 `<|eom|>`). 역할 `system/user/assistant/tool` . assistant는 `to=user` / `to=self` (reasoning) / `to=<tool_name>` .

- 특수 토큰: `<|begin_of_text|>`, `<|end_of_text|>`, `<|start|>`, `<|message|>`, `<|eot|>`, `<|eom|>`, `<|image|>`.
- `reasoning_strength`: `xhigh|high|medium|low`, 기본 `high`. 이전 reasoning은 `reasoning_content`로 재생 가능.
- 툴: OpenAI function schema를 `tools`로 전달 → 모델은 ATEM(`<atem:function_calls>`)로 출력, 결과는 `tool` 메시지(`<tool_output>`). vLLM/llama.cpp가 표준 `tool_calls`로 파싱.
- 이미지는 `AutoProcessor` 사용 (bare tokenizer 금지).
- 샘플링 권장: `temperature 1.0`, `top_p 0.95`, `top_k 64`.

한계/주의

- **툴 호출은 턴당 1개, parallel tool calls 미지원.**
- 기본 컨텍스트 128K. 재생된 reasoning도 컨텍스트에 포함 → 긴 agentic loop는 prune/summarize.
- `max_tokens` 넉넉히, 긴 reasoning은 streaming으로 timeout 회피.

인용

- "Muse Glimmer supports one tool call per turn. It does not support parallel tool calls"
- "Muse Glimmer routinely produces multi-thousand-token chains of thought, and its default context window is 128K tokens."

---

6. https://dev.meta.ai/docs/muse-glimmer/spec-decode

요약

- DFlash = Glimmer의 block-diffusion speculative decoding. 소형 draft가 제안, Glimmer가 검증. 긴 reasoning trace에서 효과 큼.
- Draft 모델 2종: `meta-models/Muse-Glimmer-30B-assistant` (변환 불필요, SGLang bf16/GGUF/NVFP4), `dflash-Muse-Glimmer-30B-Q4_K_M.gguf` (~1.6GB, `…-30B-GGUF` 레포, llama.cpp/ExecuTorch/SGLang GGUF path).
- llama.cpp: `-md <draft.gguf> --spec-type draft-dflash` + draft 플래그.
- SGLang: `--speculative-algorithm DFLASH` + draft path + block size. MLX 백엔드 미지원.
- ExecuTorch: `export_dflash`로 target+draft 동시 export (draft는 embedding/output head 공유).

한계/주의: 속도 수치·완전한 명령 예시 없음("Run inference" 페이지 참조). draft 메모리 headroom 필요. 양자화와 병용 가능.

인용

- "more than one token can land per verification step"
- "Each runtime exposes its own draft-length and block-size knobs"

---

7. https://dev.meta.ai/docs/muse-spark-open/prompting

요약

- 역할 `system`(또는 `developer`)/`user`/`assistant`/`tool`. 사용자 system prompt는 템플릿 기본 system 블록 뒤에 붙음 (기본 문구 → 사용자 프롬프트 → tool 정의 → reasoning strength → recipient 목록). 날짜는 `chat_template_kwargs.current_date`.
- 특수 토큰: `<|begin_of_text|>`, `<|start|>`, `<|message|>`, `<|eot|>`, `<|eom|>`; 미디어 `<|patch|>`(이미지), `<|video|>`, `<|audio|>`. 툴은 `<atem:function_calls>` / `<atem:invoke>` / `<atem:parameter>` / `<tool_output name=…>`.
- Recipient: `self`(reasoning), `commentary`(진행 보고), tool 이름, `user`.
- `reasoning_effort`: `low`(8) / `medium`(32) / `high`(256) / `xhigh`(512, 기본). `none`, `max` 등은 거부. system prompt의 `Reasoning strength: <number>` 줄이 override.
- 툴: OpenAI function-calling 형식, **parallel calls 지원** (Glimmer와 다름). 결과는 `tool` 메시지 + `tool_call_id`.
- 멀티모달: `image_url` / `video_url` / `audio_url` parts를 user 메시지에.

한계/주의

- reasoning 끌 수 없음. 높은 strength는 streaming 필수.
- `<|eom|>`을 custom stop token으로 쓰지 말 것. `apply_chat_template`에 tool `arguments`는 dict로.
- `tool_call_id`는 렌더되지 않음 → 같은 툴 여러 번 호출 시 호출 순서대로 결과 반환.
- 짧거나 저해상도(~100px) 비디오는 "볼 수 없다"고 답할 수 있음 → 수 초 이상, 짧은 변 448px+.
- 컨텍스트 길이/샘플링/rate limit은 이 페이지에 없음 (Recommended settings 참조).

인용

- "Muse Spark Open always reasons before it answers."
- "Higher strengths produce long reasoning, so request streaming (`stream: true`) to avoid timeouts."

---

8. https://dev.meta.ai/docs/pricing-rate-limits

가격 (per 1M tokens, Standard / Contributor)

모델	cached input	input	output
`muse-wonder-1.0`	$0.20 / n/a	$4.00 / n/a	$20.00 / n/a
`muse-spark-1.3` / `1.2` / `1.1`	$0.15 / $0.002	$1.25 / $0.10	$4.25 / $0.20

- Web search grounding $2.50 / 1,000 쿼리 (토큰 비용 별도). `muse-image-1.0` $0.01/장 (실패·필터는 과금 안 함). Voice Transcribe $0.18/시간. SAM 3.1 $2.50/1,000 이미지, $0.20/1,000 프레임(요청당 최소 50프레임).

Rate limits

모델	RPM	TPM
`muse-wonder-1.0`	3,000	4,000,000
`muse-spark-1.3` / `1.2` / `1.1`	3,000 (각)	4,000,000 (각)
`muse-spark-1.3-contributor` / `1.2-contributor`	150 (각)	3,000,000 (각)
`sam-3.1`	3,000	없음
`muse-image-1.0`	150	없음
`muse-voice-transcribe-1.0`	없음	없음 (256 동시 스트림, 16,000 스트림/시간)

- Background responses(`background: true`): 팀당 분당 600건 신규 제출, 위 한도와 별개.

스코프: **팀 단위**. API key별이 아니고 팀 내 모든 키가 공유. 모델별로 독립.

Spend limits / usage tiers / 대시보드: 이 페이지에 **없음**.

모니터링: 토큰 기반 응답마다 `x-ratelimit-limit-tokens` , `x-ratelimit-remaining-tokens` , `x-ratelimit-limit-requests` , `x-ratelimit-remaining-requests` 헤더. 응답 `usage` 객체(`cached_tokens` 포함). 주입된 system prompt 토큰은 과금·usage 계산 제외. 이미지 응답 토큰 수는 참고용.

Contributor mode: 할인/추가 사용량 대신 콘텐츠(prompt, code, files, context, 응답) 사용 권한 제공, 향후 Meta 모델 학습에 쓰일 수 있음. Model API에서는 `muse-spark-1.3-contributor` , `1.2-contributor` 만. Wonder/Voice Transcribe에는 없음. Standard 모드 콘텐츠는 학습에 안 씀. Muse Code 구독은 기본 Contributor(설정에서 토글).

초과 시: 요청 거부(429). 사용량이 한도 아래로 떨어지면 재개. exponential backoff + jitter (~500ms부터), `Retry-After` 있으면 그 이상 대기. Voice realtime 스트림 거부는 code 1013으로 close, 파일 요청은 429, transcribe 응답엔 `x-ratelimit-*` 없음.

인용

- "Contributor mode offers discounted pricing or more usage in exchange for permission to use your content"
- "When you exceed a limit, the API rejects the request."

---

9. https://dev.meta.ai/docs/authentication

요약

- 키 발급: Model API dashboard → API keys 탭 → 이름 지정 → 생성. 전체 키는 **한 번만** 표시.
- 형식: `LLM_` + 숫자 ID + secret (언더스코어 구분). `Authorization: Bearer <key>` , 예시 엔드포인트 `https://api.meta.ai/v1/chat/completions` , 권장 env var `MODEL_API_KEY` .
- SDK: OpenAI SDK, Anthropic SDK, LangChain, LiteLLM 호환. 단 `MODEL_API_KEY` 를 자동으로 읽지 않으므로 명시 전달.
- **여러 키 발급 가능**. 앱/개발자당 1키 권장(독립 revoke). 한 키 삭제가 다른 키에 영향 없음.
- 로테이션: 새 키 생성 → 앱 이전 → 구 키 삭제.
- 키는 **팀 소속**, 같은 팀 키는 rate limit 공유.
- **Project** 개념·키별 permission scope·키별 한도: 이 페이지에 **없음**.

한계/주의: 클라이언트(모바일/브라우저)에 키 노출 금지 → 서버 프록시. 유출 키는 팀 한도를 소모하고 업로드 파일 접근 가능. 키 하나 폭주하면 팀 전체 영향.

인용

- "Copy the key right away. You only see it once."
- "Keys on the same team share rate limits."

---

10. https://dev.meta.ai/docs/token-counting

요약

- 엔드포인트 `POST https://api.meta.ai/v1/responses/input_tokens` (Bearer). Anthropic 호환 `POST /v1/messages/count_tokens`.
- 요청은 `/v1/responses` 와 같은 필드, 최소 `model` + `input`. `previous_response_id` 이력, instructions, tool 정의, 미디어, 스캐폴딩까지 반영해 카운트.
- 응답 `{"object": "response.input_tokens", "input_tokens": N}` (Anthropic 형식은 `{"input_tokens": N}`).
- 모델별 tokenizer + 프롬프트 템플릿 적용 → 해당 모델에 정확. 로컬 tokenizer 라이브러리 비권장.
- 카운트에 Meta hidden system prompt, 기본 developer message, built-in tool 스캐폴딩 포함.
- usage 필드: Responses = `input_tokens` / `output_tokens` / `total_tokens` / `input_tokens_details.cached_tokens` / `output_tokens_details.reasoning_tokens`; Chat Completions = `prompt_tokens` / `completion_tokens` / `total_tokens`. 스트리밍 Chat Completions는 `stream_options: {"include_usage": true}`.

한계/주의: 생성·툴 실행·usage 기록 안 함 → 과금 견적 아님. hosted tool 사용 시 `usage.input_tokens` 는 반복 합산값이라 컨텍스트 점유량으로 쓰지 말 것. 이미지/파일 비용은 모델·해상도별로 달라 엔드포인트로 측정. 검증 실패는 400. Conversation 리소스 미지원. 숫자 한도 없음.

인용

- "Tokenizers differ per model, so the same text can map to different totals on different models."
- "Do not use `usage.input_tokens` as a final-context occupancy value."

---

11. https://dev.meta.ai/docs/error-handling

요약

- 오류 형식 `{"error": {"message", "type", "param", "code"}}`. `type`: `invalid_request_error` (4xx), `authentication_error` (401), `rate_limit_error` (429), `server_error` (5xx), `billing_error` (402). `code` 는 validation 오류에서 종종 `null` → `type` +status로 분기.
- 상태 코드: 400 (재시도 X), 401 `invalid_api_key`, 402 `billing_not_configured`, 403 모델/기능 접근 없음, 404 `model_not_found` / `file_not_found`, 413 `payload_too_large` (background body >1MiB), **429 `rate_limit_exceeded` (팀 RPM/TPM 초과, backoff 재시도)**, 500 (재시도), 503 `server_shutting_down` / `service_overloaded` / `backend_unavailable` (재시도), 504 `gateway_timeout` (그대로 재시도 X).
- 429: `Retry-After` 헤더 포함, 그만큼 대기. 예시 메시지 "Rate limit exceeded. Please retry after 15 seconds."
- 재시도: exponential backoff + jitter, 0.5-1s 시작, 두 배씩, 3-5회 제한, 모든 오류 로깅. 503 `service_overloaded` / `backend_unavailable` 은 `Retry-After: 60`.
- 400/401/402/403/404/413은 재시도 금지.

한계/주의: 504는 `background: true`, streaming, 요청 축소로 대응. 스트림 중 실패는 `response.failed` / `error` SSE 이벤트. `input_tokens + max_output_tokens` 가 컨텍스트 안에 들어야 하고 Responses API `max_output_tokens` 최소 16. 미지 경로 404는 빈 body. 이 페이지는 rate limit 헤더 이름을 적지 않음(8번 참조).

인용

- "Rate limit exceeded. Please retry after 15 seconds."
- "Do not retry 400, 401, 402, 403, 404, or 413 errors: these require fixing the request"

---

12. https://dev.meta.ai/docs/open-weight-models

요약

- 오픈웨이트 2종: **Muse Spark Open** (816B MoE / 42B active, text·image·audio·video 입력, BF16 + NVFP4 체크포인트, agentic/coding 지향), **Muse Glimmer** (30B 멀티모달, Spark에서 distill, 로컬/엣지용).
- 다운로드는 각 모델의 "Get the model" 페이지. 서빙은 SGLang/vLLM 등 자기 하드웨어, 또는 서드파티 inference provider.
- 라이선스, 하드웨어 수치, 호스티드 파인튜닝 언급 **없음**.

인용

- "Ships as BF16 and NVFP4 checkpoints."
- "You can also call open-weight models through third-party inference providers instead of running them yourself."

---

13. https://dev.meta.ai/docs/muse-glimmer/get-the-model

요약

- Hugging Face에서 무료 다운로드, API key·gated access 없음, 다운로드 후 오프라인 동작.

- 전제: Python 3.10+, `pip install -U huggingface_hub`, `transformers` 5.15+ (미만이면 `trust_remote_code` 요구).
- 레포 4개: `meta-models/Muse-Glimmer-30B` (bf16 safetensors ~60GB, vLLM/SGLang), `…-30B-GGUF` (양자화 + vision projector + draft, llama.cpp/ExecuTorch/SGLang), `…-30B-assistant` (DFlash draft), `…-30B-ExecuTorch-PTE` (16개 export).
- GGUF: `Muse-Glimmer-30B-KQuant-17GB-Q4_K_M.gguf` ~17GB (24GB VRAM 미만), `…-Dynamic-Q4_K_XL.gguf` ~20GB (32GB VRAM), `mmproj-…-Q4_K_M.gguf` ~1.4GB (이미지 입력 필수), `dflash-…-Q4_K_M.gguf` ~1.6GB.

명령

```
hf download meta-models/Muse-Glimmer-30B --local-dir ./muse-glimmer-30b
hf download meta-models/Muse-Glimmer-30B-GGUF --local-dir ./muse-glimmer \
  --include "Muse-Glimmer-30B-KQuant-17GB-Q4_K_M.gguf" \
  --include "mmproj-Muse-Glimmer-30B-Q4_K_M.gguf"
hf download meta-models/Muse-Glimmer-30B-assistant --local-dir ./muse-glimmer-assistant
```

한계/주의: ExecuTorch 레포는 전체 받지 말고 `--include`로 export 하나만. 라이선스·bf16 하드웨어 요구·성능 수치 없음. 검증은 tokenizer 로드 + `apply_chat_template` 렌더로.

인용

- "Everything runs offline once the weights land: there's no API key and no gated access."
- "Earlier versions lack the native Muse Glimmer path and prompt for `trust_remote_code`."

질문별 답변

(a) per-user / per-project 토큰·지출 한도를 설정할 수 있나?

문서상 불가.

- Rate limit은 **팀 단위** 고정값(모델별 RPM/TPM)이고 API key별로 쪼갤 수 없음 (pricing-rate-limits, authentication: "Keys on the same team share rate limits.").
- **Spend limit, usage tier, 대시보드, project 스코핑, 키별 permission scope** 모두 문서에 없음 (pricing-rate-limits, authentication).
- 제공되는 것: 여러 키 발급(앱/개발자당 1키, 독립 revoke), 응답의 `x-ratelimit-*` 헤더와 `usage` 객체, `/v1/responses/input_tokens` 카운트 엔드포인트.

앱이 직접 구현해야 하는 것

- 서버 프록시 뒤에서 end-user/프로젝트별 예산 원장: 요청 전 `input_tokens` 엔드포인트로 사전 카운트 → 응답 `usage`(`input_tokens`, `output_tokens`, `cached_tokens`, `reasoning_tokens`)를 사용자별로 누적 → 8번 페이지 단가로 비용 환산 → 임계치 초과 시 앱 레벨 거부.
- 팀 TPM(4M)/RPM(3k) 공유이므로 앱 자체 per-user rate limiter(토큰 버킷)로 한 사용자의 폭주가 팀 전체 429를 유발하지 않게 할 것.
- `x-ratelimit-remaining-*` 헤더를 읽어 팀 잔여량이 낮으면 선제적으로 throttle.
- 앱/환경(dev/prod)별 키를 분리해 유출 시 독립 revoke — 단 한도는 여전히 팀 공유라 격리는 안 됨.

(b) Glimmer / Spark Open 파인튜닝·포스트트레이닝 경로

Muse Glimmer 30B (muse-glimmer/post-training)

- SFT: Axolotl 0.19.0 (QLoRA, bf16 LoRA FSDP2, full bf16 FSDP2), TRL+PEFT (LoRA/QLoRA).
- RL: TitanRL/TorchTitan (GRPO, verifiable reward), TRL (DPO, GRPO).
- 데이터: OpenAI-style `messages` (이미지 인라인), JSONL, DPO `prompt/chosen/rejected`, GRPO prompt+reward fn.
- 하드웨어: 8×H100 검증, QLoRA는 1×H100 (peak 27GB). Full은 97GB/GPU peak + ~222GB 디스크. 128K ctx.

Muse Spark Open 816B-A42B (muse-spark-open/post-training)

- 오픈소스: TorchTitan (LoRA/full SFT text·image·audio, TitanRL DAPO; **Blackwell SM100+ 필수**, LoRA 16×GB300, RL 256+16 GPU), Miles (GRPO, 64×H200), NeMo AutoModel (HF 직접), Megatron Bridge, TRL SFTTrainer LoRA (64×H100).
- 데이터: JSONL chat `messages`; Miles는 DAPO-Math-17k / Geometry3K.
- 스토리지: GPU당 ~1.6TB 체크포인트급. 해커톤 규모에서는 사실상 호스티드 또는 Glimmer로 가야 함.

(c) Meta Model API에 호스티드 파인튜닝이 있나?

Meta 자체 호스티드 파인튜닝은 문서에 없음. `api.meta.ai`는 추론 API만 다루며 (open-weight-models, pricing-rate-limits 모두 미언급), 파인튜닝 모델을 Model API에서 서빙하는 경로도 없음. 대신 Spark Open 포스트트레이닝 페이지가 **서드파티 호스티드**를 안내함: Together AI (SFT, GRPO; preview), Fireworks AI (serverless LoRA SFT/DPO/ORPO/distillation/RL, rank ≤64, 동시 8 run; 학습된 LoRA는 전용 deployment에서만 서빙). Glimmer에 대한 호스티드 옵션은 페이지에 없음.

(d) Rate limit 동작과 429

- 모델별 RPM/TPM 고정(Spark 3,000 RPM / 4M TPM, contributor 150 RPM / 3M TPM), **팀 단위**로 모든 키가 공유, 모델별 독립 (pricing-rate-limits).
- 초과 시 요청 거부 → HTTP **429**, `type: rate_limit_error`, `code: rate_limit_exceeded`, `Retry-After` 헤더 포함 (error-handling). 사용량이 한도 아래로 내려가면 자동 재개.
- 재시도: `Retry-After` 만큼(이상) 대기, 없으면 exponential backoff + jitter 0.5-1s 시작, 3-5회 제한 (error-handling, pricing-rate-limits).
- 매 성공 응답의 `x-ratelimit-limit/remaining-tokens|requests` 로 잔여량 모니터링 (pricing-rate-limits).
- 별도 429 원인: background 제출 600/min/팀 초과, Voice Transcribe 파일 요청 스트림 한도 초과(realtime은 code 1013 close).

건너뛴 것: run-locally 하위의 Mac 가이드·"Run inference"·"Recommended settings"·Spark Open "Get the model" 페이지는 요청 목록에 없어 fetch하지 않음. 4개 페이지(cookbook/post-training, run-locally, open-weight-models, get-the-model)는 identity 오버레이 때문에 본문이 일부 가려졌을 수 있음. 리스크: 가격·한도 수치는 2026-10-28 fetch 기준이며 WebFetch 요약 모델을 거친 값이라 발표 자료에 쓰기 전 원문 재확인 권장.

부록 C. 해커톤 자료·Muse Code·파트너·과거 수상작 — 상세 노트

(조사일: 2026-10-28, 행사 진행 중 / 모든 인용은 원문 영어 유지)

0. 한 줄 요약

Meta가 이 해커톤에서 밀고 있는 것은 **"Muse Spark(또는 open-weight)를 두뇌로 삼고, 감사 가능한(auditable) 하네스로 감싼 장기 실행 에이전트"**다. 공식 리소스(docs, cookbook, Muse Code, 파트너 페이지)가 일관되게 agent loop · tool calling · multi-agent orchestration · computer use · 검증(verifiable) · 샌드박스/승인 패턴을 가르친다. 심사 기준(Impact 30 / Meta tooling 창의적 사용 25 / Demo 25 / Technical depth 20)은 공식 페이지에는 없고, 참가자용 가이드(로그인 필요)를 옮긴 서드파티 저장소에서 확인됐다.

---

1. 공식 해커톤 페이지 (Cerebral Valley)

- URL: https://cerebralvalley.ai/e/meta-ai-developer-hackathon (Markdown 버전: https://cerebralvalley.ai/e/meta-ai-developer-hackathon.md)
- 기간: Oct 26 12:00 PM - Nov 4 9:00 PM ET, 원격·전세계, 상금 $1M, 최대 20,000명.
- **테마 (둘 중 하나 필수)**
  - Build for Your World: 자기 도메인의 문제. 인용: "Bring a problem you know firsthand, or an idea that feels out of reach."
  - Push the Frontier: software, robotics, manufacturing, energy, materials 등.
  - 인용: "Show us what becomes possible when these models meet the real world."
- **트랙과 심사 기준(공개된 문구)**
  - Indie Developer: "Judged on creativity, technical ambition, and whether it works."
  - Startup: "Judged on technical execution and whether it holds up in production."
- **제공 모델/도구**: Muse Spark series(Model API), Muse Wonder, Muse Image, SAM 3.1, open-weights(Muse Spark Open, Muse Glimmer), Muse Code.
- **파트너 크레딧**: Baseten $100 + 2M TPM, Modal $100(+serverless GPU/sandbox/training), Fireworks $100(페이지상) + Sinan Ozdemir 워크숍.
- **Resources 섹션: 공개 페이지에는 없음.** 참가자 가이드는 로그인 필요: https://cerebralvalley.ai/e/meta-ai-developer-hackathon/details (Discord 초대, $150 크레딧 신청 폼, 제출 포털, 라이브스트림 링크가 여기 있음 — 직접 확인 불가). Waiver: https://cerebralvalley.ai/e/meta-ai-developer-hackathon/waiver
- 페이지에 링크된 파트너 문서: https://modal.com/docs/guide/endpoints , https://modal.com/docs/guide/gpu , https://modal.com/docs/guide/sandboxes , https://modal.com/docs/guide/training-quickstart

참가자 가이드 내용(서드파티 저장소 경유 — 공식 원문 아님, 검증 필요)

출처: https://github.com/meta-ai-hackathon/meta-hack (`info/grading criteria.md` , `info/event guide.md` , `info/links.md` , `info/stack and credits.md`)

- **Pass/fail 게이트**: 테마 적합 + Meta Model API 또는 open-weight 모델 실사용. 인용: "Fail either = disqualified, regardless of quality."
- **점수 루브릭(0-10, 가중)**: Impact 30% / Meta AI tooling(기본 연동 이상의 창의적 사용) 25% / Demo(3분 영상 품질) 25% / Technical depth 20%. 인용: "Use these weights, not gut feel." 트랙별 상위 5팀이 라이브 파이널(같은 루브릭).
- **보너스 ($50K)**: Most Viral Project $20K(X 게시물 반응, Nov 8까지), Best Build on Open Weight Models $10K(Glimmer/Spark Open을 자체 하드웨어 또는 파트너 추론), Community Vote $10K, Best Agentic Use-Case $5K("strongest agent autonomy and multi-step workflows"), Best Creative Build $5K(Muse Image/Wonder).
- **제출 요건**: Nov 4 9 PM ET, CV 플랫폼. 3분 데모 영상이 주 심사물, 공개 저장소, 기여자 전원 명시, 팀 최대 4명·1회 제출, **행사 중 만든 것만 시연**(불명확하면 즉시 실격).
- 상금 구조(madrobot 블로그 기준): Startup $550K(1위 $250K), Indie $400K(1위 $150K). https://madrobot.blog/2026/10/03/meta-global-ai-developer-hackathon-1-million-prizes-muse-model-api/

---

2. Meta 자체 발표

- **Meta 측 이벤트 페이지**: https://dev.meta.ai/events/global-hackathon — 내용은 CV 페이지와 동일(테마·트랙·일정). 리소스 링크: https://dev.meta.ai/docs/ , https://dev.meta.ai/docs/cookbook , https://dev.meta.ai/docs/muse-glimmer/ , https://dev.meta.ai/docs/overview#muse-spark-open , https://github.com/meta-models , https://huggingface.co/meta-models , https://muse.ai/platform (Muse Connectors)
- **Meta Connect 2026 recap** (최초 발표): https://developers.meta.com/blog/meta-connect-recap/ — 인용: "Build on Meta Model API, Muse Code or our open-weights models." MSL 엔지니어가 10일간 워크숍 진행. Muse Code에 대해: "Muse Code turns that intelligence into a multi-agent machine built for long-horizon engineering tasks."

- **X 게시물** (직접 fetch는 403; 검색 스니펫으로 확인)
  - https://x.com/MetaforDevs/status/2110045152601633114 — "Build the future of superintelligence with us… Each selected participant gets $150 in Meta Model API credits."
  - https://x.com/MetaforDevs/status/2112971571321332179 — "Open weights, meet your favorite stack. We're teaming up with @FireworksAI_HQ, @baseten and @modal…"
- Threads 게시물·별도 ai.meta.com 블로그 글: **찾지 못함.**
- 보도 요약: https://aimagazine.blog/meta-opens-its-1m-global-ai-developer-hackathon-20000-seats-150-api-credits-each-and-applications-close-october-25/ , https://superpowerdaily.com/posts/meta-sets-october-26-start-for-1-million-ai-hackathon-capped-at-20-000-builders

---

3. Meta Model API Quickstart / SDKs

- Quickstart: https://dev.meta.ai/docs/quickstart
  - Base URL `https://api.meta.ai/v1` (OpenAI 형식), `https://api.meta.ai` (Anthropic Messages 형식). 키는 `MODEL_API_KEY`, Bearer.
  - 인용: "Model API is drop-in compatible with the OpenAI SDK, the Anthropic SDK, and OpenAI-compatible agent CLIs."
  - 기본 모델 `muse-spark-1.3` (context 1,048,576 / output 131,072). 엔드포인트: Responses, Chat Completions, Messages. 예제: Python/TS(OpenAI SDK), requests, curl.
  - Quickstart가 **첫 경로로 Muse Code CLI 설치**를 권장. OpenCode 설정에서 `include: ["reasoning.encrypted_content"]` 가 "what carries Muse Spark's reasoning across turns."
  - 주의: "The OpenAI SDK doesn't auto-read `MODEL_API_KEY`" — 명시 전달 필요. Reasoning 토큰은 output_tokens에 과금.
- SDKs: https://dev.meta.ai/docs/sdks
  - 인용: "Meta Model API doesn't need a client of its own." / "Build with the clients you already run."
  - OpenAI SDK(Py/TS), Anthropic SDK(Py, `auth_token` 로 키 전달, 첫 블록은 `redacted_thinking`), LiteLLM(`openai/` prefix), LangChain/LlamaIndex/Vercel AI SDK. SAM 클라이언트: `@meta-sam/parser`, `meta_sam_parser`, `@meta-sam/graphics|video|react`.
- 관련 가이드: Coding agents https://dev.meta.ai/docs/coding-agents (OpenCode·Codex·Claude Code·Goose/Roo 설정; Claude Code는 `ANTHROPIC_BASE_URL=https://api.meta.ai`, 모델 `muse-wonder-1.0`), Agent frameworks https://dev.meta.ai/docs/agent-frameworks (Claude Agent SDK, Codex app-server), Computer use https://dev.meta.ai/docs/computer-use (인용: "The computer tool defines the action protocol, not an environment." / "a disposable VM or container keeps the agent off your machine.")

---

4. Muse Code

- Overview: https://dev.meta.ai/docs/muse-code
  - Meta의 터미널/CI용 코딩 에이전트, Muse Spark용으로 설계. 설치 `curl -fsSL https://dev.meta.ai/install.sh | sh`. 기본 모델 `muse-spark-1.2`.
  - 구성 요소: subagents, skills, hooks, MCP, workflows, session messaging. `muse exec "<prompt>"` 는 "runs one prompt to completion for scripts and CI."
  - SDK: "you can drive a Muse Code session from your own program over a stable, versioned session protocol." (`@muse-code/sdk`, `muse serve`, `muse schema`; docs https://meta-models.github.io/muse-code-sdk , repo https://github.com/meta-models/muse-code-sdk)
  - 선택 기준: "Call the API directly when you build your own agent or app." — 자체 앱/에이전트는 API, 코딩 작업 자동화는 Muse Code.
- Workflows: https://dev.meta.ai/docs/muse-code/workflows
  - 여러 child agent를 병렬/단계별로 실행하는 orchestration. 인용: "Muse Code creates the orchestration, runs the child agents, and returns a synthesized result to your session."
  - 정의는 **JavaScript 파일** (`.agents/workflows/`, `muse workflows save <name> --from ./x.js --scope project`), 최대 1,000 child, 동시 16. 설정 `auto|explicit|off`.
  - **제약**: 공개 `aarch64-apple-darwin` 패키지에는 workflow 엔진 미포함(staged rollout). 해커톤 프로젝트에 넣을 때 환경 확인 필요.
- Extending: https://dev.meta.ai/docs/muse-code/extending
  - Subagents(기본 8, 1-64, 격리 worktree 옵션), 배경 observer agents(memory/skills/goals/verification), Skills(`/plan`, `/grill`, `/taste`, `/threejs`; "you can add your own or import them from other agents."), Hooks(shell 명령을 lifecycle 이벤트에 바인딩; "Only add hooks whose commands you've read."), MCP(stdio/streamable_http, OAuth; "Only connect servers you trust."), Headless(`muse exec`, JSONL 출력, `--yolo` 는 일회용 환경에서만).
- Cookbook (Muse Code 섹션): https://dev.meta.ai/docs/cookbook/muse-code — 인용: "the harness building blocks for safety, recovery, and orchestration." 10개 레시피: audit-agent-sessions, deterministic-replay(CI 골든 픽스처), staged-approvals, contained-execution, immutable-

guardrails, subagent-fanout, goal-tracking("the harness audits the work against acceptance checks before the goal can close."), bundled-skills, loop-and-cron, side-chats. (각 URL: `https://dev.meta.ai/docs/cookbook/<slug>`)

- 블로그 "Build with Muse Code": https://dev.meta.ai/resources/blog/build-with-muse-code/ — 인용: "every subagent it spawns, every tool call, every steer and cancel, is observable and replayable through the event log." / "each child gets its own git worktree, so parallel children never collide on the same files." 예제 repo `bastion_breaker`: https://github.com/meta-models/meta-model-cookbook/tree/main/04_muse_code/06_subagent_fanout/bastion_breaker
- **프로젝트에서의 활용 방식(문서 기준)**: (a) 개발 도구로 — `/plan`·fan-out·goal tracking으로 빌드 속도; (b) 제품 구성요소로 — `muse exec`/SDK로 headless 실행, hooks·staged approvals·sandbox·event log로 "auditable agent" 데모; (c) 자동화 — loop/cron, CI replay.

---

5. 공개 Cookbook / 예제 저장소

- GitHub org https://github.com/meta-models (공개 repo 5개)
  - https://github.com/meta-models/meta-model-cookbook — 7개 섹션: 01_api_fundamentals, 02_agent_patterns, 03_use_cases, 04_muse_code, 05_muse_image, 06_muse_voice, 07_sam. Colab + 독립 Python. 기본 모델 `muse-spark-1.3`.
  - https://github.com/meta-models/meta-oss-cookbook — open-weight 모델 레시피(Apache-2.0).
  - https://github.com/meta-models/legal-agent-workshop — Muse Code + Muse Spark로 법률 에이전트 구축 워크숍. Session 1: basic agent loop → "ends with an unverified memo."; Session 2: 검증 레이어(ground truth 대조, refusal/escalation, attorney sign-off). **"verifiable agent" 패턴의 공식 교재** — Agents 102 주제와 일치.
  - https://github.com/meta-models/muse-code-sdk , https://github.com/meta-models/meta-sam
- 문서 cookbook 인덱스: https://dev.meta.ai/docs/cookbook — 인용: "Recipes for building with Muse models, from your first API call to complete solutions."
  - Agent patterns(https://dev.meta.ai/docs/cookbook/agent-patterns): basic-agent-loop, interleaved-reasoning-tool-use, multi-turn-context-management, search-and-replace-edits, alert-fatigue-copilot. 인용: "Turn a model into an agent. These loops handle planning, self-correction, and staying coherent across long runs."
  - Use cases: perception-chart-analysis, perception-error-screenshot-fix, rayban-openclaw(스마트글래스), generating-slides, browser-verified-web-design, iterative-game-dev, sandboxed-execution, multi-agent-orchestration, one-shot-game-dev, perception-grounding, github-agent, computer-use, computer-use-macos, glimmer-api-agent(로컬 에이전트가 먼저 묻기).
  - 대표 레시피 2개:
    - Multi-agent orchestration (Hermes, 4 role: pm/backend/frontend/gtm, Kanban 코멘트로 협상): https://dev.meta.ai/docs/cookbook/multi-agent-orchestration — 인용: "every cross-role decision is a comment on a Kanban task, never a back-channel chat, so the run is replayable…" / "asymmetry is what keeps the orchestration honest." 코드: https://github.com/meta-models/meta-model-cookbook/blob/main/03_use_cases/08_multi_agent_orchestration/README.md
    - GitHub agent (OpenCode + GitHub Actions + Model API, 읽기전용→label-gated 수정): https://dev.meta.ai/docs/cookbook/github-agent — 인용: "OpenCode is the runtime, the Meta Model API is the brain, GitHub is the surface" / "every code change lands as a PR for human review". 코드: https://github.com/meta-models/meta-model-cookbook/blob/main/03_use_cases/11_github_repo_agent/README.md
- 블로그 "Build with Muse Spark": https://dev.meta.ai/resources/blog/build-with-muse-spark/ — 권장 패턴: agent에 객관적 체크(failing test) 주기, Responses API + function tools + `previous_response_id`, `reasoning_effort` 조절, `developer` 메시지 사용, 내장 `web_search` 툴, computer use("The desktop runs in a throwaway sandbox, so the model never touches your machine").
- 블로그 "Build with Muse Spark Open": https://dev.meta.ai/resources/blog/build-with-muse-spark-open — 인용: "trained for long-horizon agentic work such as coding and customer service agents"; 호스팅 파트너·post-training(TorchTitan, Fireworks serverless LoRA, Together SFT/GRPO, TRL) 링크 다수(아래 URL 목록).

---

6. 라이브 세션 (Agents 101 / 102 / Fireworks)

- **공식 녹화·슬라이드: 공개 웹에서 찾지 못함.** YouTube/X/Threads 검색 결과 없음. https://dev.meta.ai/resources/videos/ 에는 Llama 영상만 있음.
- 참가자 저장소에 적힌 일정(비공식, 서로 일치):
  - Tue Oct 27 12-1 PM ET — **Agents 101**, Suraj Subramanian, "Build a multimodal AI agent"
  - Wed Oct 28 12-1 PM ET — **Agents 102**, Yash Khare, "Best practices for verifiable agents"
  - Fri Oct 30 12-1 PM ET — **Fireworks**, Sinan Ozdemir, "Specialized intelligence on Fireworks (Muse Open models)" — "inference and training" 제목은 공식 확인 안 됨(그의 Fireworks 직함 "fast inference and training"에서 유래한 듯).
  - 출처: https://github.com/meta-ai-hackathon/meta-hack (`info/event guide.md`), https://github.com/kopias/sous — 인용: 세션은 "recorded to #resources" (Discord; "docs, credit forms, session recordings").
- 대체 자료(Connect 2026 세션, 공개): "Building with Muse Spark and Muse Code: from capabilities to production" (Claire Zhou) https://developers.meta.com/resources/videos/building-with-muse-spark-and-muse-code — 모델은 "see, decide, and act", 하네스는 "reliability, scale, and

auditability required to ship production code." 외 https://developers.meta.com/resources/videos/multimodal-intelligence-of-muse-spark , https://developers.meta.com/resources/videos/the-road-ahead-superintelligence-blueprint

---

7. 파트너 리소스

- **Baseten**: https://www.baseten.co/library/muse-spark-open/ — base_url `https://inference.baseten.co/v1` , 예제 모델 ID `meta-models/muse-spark-open` . 인용: "Muse Spark Open is a model designed specifically for coding agents." 가격 $1.25/$0.15(cache)/$4.25 per 1M. **해커톤용 ID `meta-models/muse-spark-open-1026` 은 Baseten 페이지에 없고** 참가자 가이드 전사본(meta-hack repo `info/links.md` : "The docs default to `meta-models/muse-spark-open` , which is the wrong ID for this hackathon.")과 참가 프로젝트 pocket-muse에서만 확인. 크레딧: $100 + 2M TPM, 선착 2,000명·use-case 심사.
- **Modal**: 블로그 https://modal.com/blog/meta-muse-spark-open-now-available-on-modal , 라이브러리 https://modal.com/library/meta/muse-spark-open — 모델 ID `meta-models/Muse-Spark-Open-816B-A42B` , `modal endpoint create --shared --model ...` , Shared Endpoint "OpenAI-compatible and already live". 가격 동일. Muse 전용 Modal docs 예제는 없음(일반 vLLM 예제만). 크레딧 $100, 선착 1,000명.
- **Fireworks**: https://fireworks.ai/models/fireworks/muse-spark-open — ID `accounts/fireworks/models/muse-spark-open` , context 1040k, serverless LoRA 학습은 `muse-spark-open-nvfp4` 베이스로 Serverless Training API(https://docs.fireworks.ai/fine-tuning/training-api/serverless). Glimmer: https://fireworks.ai/models/fireworks/muse-glimmer-30b , 블로그 https://fireworks.ai/blog/meta-muse-glimmer . 크레딧: CV 페이지 $100 vs 가이드 전사본 $25(shortlisted만) — 불일치.
- **Together AI**: https://www.together.ai/models/muse-glimmer — ID `meta-models/Muse-Glimmer-30B` , $0.35/$0.04/$1.50, 128K+, 인용: "built for always-on agents rather than chat alone." 단 페이지에 "Launching soon" 대기열 배너 — 실제 가용 여부 확인 필요. Spark Open: https://www.together.ai/models/muse-spark-open ; post-training https://docs.together.ai/docs/post-training/quickstart-sft , https://docs.together.ai/docs/post-training/quickstart-rl
- 기타 호스팅: Groq https://console.groq.com/docs/model/meta-models/muse-spark-open , OpenRouter https://openrouter.ai/meta/muse-spark-open , HF 가중치 https://huggingface.co/meta-models/Muse-Spark-Open-816B-A42B (Muse Model License — "Built with Muse Spark Open" 표기 의무 보도 있음, 라이선스 원문 확인 필요: https://huggingface.co/meta-models/Muse-Spark-Open-816B-A42B/blob/main/LICENSE.md)

---

8. 과거 Meta 해커톤 우승작과 패턴

- **LlamaCon Hackathon (May 2025)** https://ai.meta.com/blog/llamacon-hackathon/
  - 1st OrgLens: Jira/GitHub/문서/이력서 → knowledge graph → 전문가 매칭 + digital twin (https://github.com/TheCl3m/llama-hack)
  - 2nd Compliance Wizards: 거래 사기 탐지 + 이메일/음성 비서 + 멀티모달 뉴스 검색 (https://github.com/michaelwaves/llamacon-hackathon)
  - 3rd Llama CCTV Operator: 자연어로 이벤트 정의, fine-tuning 없이 영상 감시 (https://github.com/torayeff/llamacon-hackathon-2025-sf/)
  - Best API Usage Geo-ML: 400쪽 지질 보고서 → DSL → 3D 모델 (https://github.com/williamjsdavis/geo-lm). 인용: "I spend most of my time reading through geology documents in my day-to-day work."
  - 파이널 영상: https://www.youtube.com/playlist?list=PLb0IAmt7-GS2eX0SYxHXdKPDwTNUxXB15
- **Llama 3 Hackathon (May 2024)** https://ai.meta.com/blog/llama-3-hackathon-recap-cerebral-valley/ — 1st OpenGlass($20 스마트글래스, https://github.com/BasedHardware/openglass), 2nd Deb8(모델 토론 + 심판 에이전트), 3rd AAA(활성화 연구). 갤러리 https://metallama3.devpost.com/project-gallery
- Llama Impact Hackathon 2024: https://about.fb.com/news/2024/11/metas-llama-impact-hackathon-pioneering-ai-solutions-for-public-good/
- 2026 Llama 해커톤 우승작: **없음(찾지 못함).** 이번 해커톤 우승작은 Nov 9 발표 예정.
- **관찰되는 승리 패턴**: (1) 실제 도메인 데이터(사내 문서, CCTV, 지질 보고서)에 모델을 꽂아 사람이 매일 하는 반복 업무를 대체, (2) 멀티모달·롱컨텍스트 등 해당 릴리스의 차별 기능을 눈에 띄게 사용, (3) 물리 세계/하드웨어 연결(글래스, 카메라), (4) 멀티 에이전트(토론+심판).
- 현재 진행 중 공개 참가작(우승작 아님, 패턴 참고): pocket-muse(ESP32 음성 가젯, Spark + Muse Image + Baseten `muse-spark-open-1026`) https://github.com/SaintSinC/pocket-muse ; yebo-muse-agent(AP 에이전트, "Muse Spark proposes. Yebo authorizes", prompt-injection 방어, 89 tests) https://github.com/Unclefole/yebo-muse-agent ; musez-home(Raspberry Pi 음성, "latency is the product", p50/p95 측정) https://github.com/ztdgtll/musez-home ; sous(카메라+SAM 3.1+Spark judge로 요리 감시) https://github.com/kopias/sous

---

9. 종합: Meta가 이 해커톤에서 보상하는 것 (근거 기반)

1. **"진짜 쓰이는" 도메인 문제 + Impact 30%** — 테마 1 문구("a problem you know firsthand"), 루브릭 Impact 최대 가중, 과거 우승작 모두 실무 데이터 기반.
2. **기본 API 호출을 넘어선 Meta 도구 사용(25%)** — 루브릭 "going beyond basic integration"; 공식 cookbook이 밀어주는 것: tool calling + built-in `web_search` , Responses API `previous_response_id` /reasoning 유지, computer use(샌드박스), multimodal 입력(이미지/영상/음성, SAM 3.1), Muse Image/Wonder(Best Creative Build), open-weight 자체 서빙/파인튜닝(Best Build on Open Weight $10K, Fireworks LoRA·Together GRPO 레시피).

3. **검증 가능·감사 가능한 에이전트(verifiable, auditable)** — Agents 102 주제, legal-agent-workshop의 "unverified memo → verification layer", Muse Code cookbook 전체(audit log, deterministic replay, staged approvals, immutable guardrails, goal tracking), Connect 세션 문구 "reliability, scale, and auditability required to ship production code." Startup 트랙 기준 "whether it holds up in production"과 직결.
4. **장기 실행·멀티스텝 자율성** — Best Agentic Use-Case 보너스 "strongest agent autonomy and multi-step workflows"; Spark Open 설명 "long-horizon agentic work"; Glimmer "always-on agents rather than chat alone."; multi-agent orchestration 레시피(역할 분리 + Kanban 감사 추적).
5. **동작하는 데모(25%) + 3분 영상 + 공개 repo + 행사 중 신규 작업만** — Indie 기준 "whether it works", 제출 규칙상 데모 영상이 "primary judging artifact". 바이럴 보너스($20K)는 X 게시 전제.
6. **"사람이 승인, 모델은 제안" 안전 설계** — github-agent("never merges or approves"), staged approvals, computer-use `pending_safety_checks` ("Treat a safety check as a decision point in your harness"), yebo 참가작도 같은 프레이밍. Meta 자체 안전 보고서가 Spark의 prompt-injection 취약을 인정한 만큼, 이를 방어하는 설계는 Technical depth에서 유리.

실무 시사점(이 저장소의 order-exceptions 프로젝트 기준): Spark 1.3를 Responses API + function tools로 쓰고, 예외 처리 결정마다 감사 로그·승인 게이트를 두며(Muse Code cookbook 패턴 차용), Spark Open(Baseten `-1026`) 경로를 하나 더 붙이면 open-weight 보너스 자격도 생긴다.

---

10. 발견한 URL 전체 목록

해커톤/발표

- https://cerebralvalley.ai/e/meta-ai-developer-hackathon
- https://cerebralvalley.ai/e/meta-ai-developer-hackathon.md
- https://cerebralvalley.ai/e/meta-ai-developer-hackathon/details (로그인)
- https://cerebralvalley.ai/e/meta-ai-developer-hackathon/waiver
- https://dev.meta.ai/events/global-hackathon
- https://developers.meta.com/blog/meta-connect-recap/
- https://x.com/MetaforDevs/status/2110045152601633114
- https://x.com/MetaforDevs/status/2112971571321332179
- https://madrobot.blog/2026/10/03/meta-global-ai-developer-hackathon-1-million-prizes-muse-model-api/
- https://aimagazine.blog/meta-opens-its-1m-global-ai-developer-hackathon-20000-seats-150-api-credits-each-and-applications-close-october-25/
- https://superpowerdaily.com/posts/meta-sets-october-26-start-for-1-million-ai-hackathon-capped-at-20-000-builders
- https://nandigamharikrishna.substack.com/p/i-got-into-metas-1m-global-ai-developer
- https://github.com/meta-ai-hackathon/meta-hack (가이드 전사본)

Model API docs

- https://dev.meta.ai/docs/ , https://dev.meta.ai/docs/quickstart , https://dev.meta.ai/docs/sdks , https://dev.meta.ai/docs/models , https://dev.meta.ai/docs/pricing-rate-limits , https://dev.meta.ai/docs/authentication , https://dev.meta.ai/docs/tool-calling , https://dev.meta.ai/docs/tool-search , https://dev.meta.ai/docs/structured-output , https://dev.meta.ai/docs/reasoning , https://dev.meta.ai/docs/protocols/responses , https://dev.meta.ai/docs/protocols/messages , https://dev.meta.ai/docs/coding-agents , https://dev.meta.ai/docs/agent-frameworks , https://dev.meta.ai/docs/computer-use , https://dev.meta.ai/docs/api-reference , https://dev.meta.ai/docs/api-reference/status , https://dev.meta.ai/docs/muse-spark-open , https://dev.meta.ai/docs/muse-glimmer , https://dev.meta.ai/docs/muse-spark-open/get-the-model , https://dev.meta.ai/docs/muse-spark-open/deploy , https://dev.meta.ai/docs/muse-spark-open/prompting , https://dev.meta.ai/docs/muse-spark-open/post-training

Muse Code

- https://dev.meta.ai/docs/muse-code , https://dev.meta.ai/docs/muse-code/workflows , https://dev.meta.ai/docs/muse-code/extending , https://dev.meta.ai/docs/muse-code/permissions , https://dev.meta.ai/docs/muse-code/configuration , https://dev.meta.ai/docs/muse-code/session-messaging , https://dev.meta.ai/docs/muse-code/interactive , https://dev.meta.ai/docs/cookbook/muse-code , https://meta-models.github.io/muse-code-sdk , https://dev.meta.ai/resources/blog/build-with-muse-code/ , https://research.meta.ai/blog/introducing-muse-code-and-muse-spark-1-2

Cookbook / 예제

- https://dev.meta.ai/docs/cookbook , https://dev.meta.ai/docs/cookbook/agent-patterns , https://dev.meta.ai/docs/cookbook/api-fundamentals , https://dev.meta.ai/docs/cookbook/use-cases , https://dev.meta.ai/docs/cookbook/basic-agent-loop , https://dev.meta.ai/docs/cookbook/tool-function-calling , https://dev.meta.ai/docs/cookbook/multi-agent-orchestration , https://dev.meta.ai/docs/cookbook/github-agent , https://dev.meta.ai/docs/cookbook/computer-use , https://dev.meta.ai/docs/cookbook/sandboxed-execution , https://dev.meta.ai/docs/cookbook/browser-verified-web-design , https://dev.meta.ai/docs/cookbook/glimmer-api-agent , https://dev.meta.ai/docs/cookbook/subagent-fanout , https://dev.meta.ai/docs/cookbook/goal-tracking , https://dev.meta.ai/docs/cookbook/deterministic-replay , https://dev.meta.ai/docs/cookbook/staged-approvals , https://dev.meta.ai/docs/cookbook/immutable-guardrails , https://dev.meta.ai/docs/cookbook/audit-agent-sessions , https://dev.meta.ai/docs/cookbook/loop-and-cron , https://dev.meta.ai/docs/cookbook/muse-spark-open-mac

- https://github.com/meta-models , https://github.com/meta-models/meta-model-cookbook , https://github.com/meta-models/meta-oss-cookbook , https://github.com/meta-models/legal-agent-workshop , https://github.com/meta-models/muse-code-sdk , https://github.com/meta-models/meta-sam , https://github.com/meta-models/meta-model-cookbook/tree/main/04_muse_code/06_subagent_fanout/bastion_breaker , https://github.com/meta-models/meta-model-cookbook/blob/main/03_use_cases/08_multi_agent_orchestration/README.md , https://github.com/meta-models/meta-model-cookbook/blob/main/03_use_cases/11_github_repo_agent/README.md
- https://dev.meta.ai/resources/blog/build-with-muse-spark/ , https://dev.meta.ai/resources/blog/build-with-muse-spark-open , https://huggingface.co/meta-models , https://huggingface.co/meta-models/Muse-Spark-Open-816B-A42B , https://huggingface.co/meta-models/Muse-Spark-Open-816B-A42B/blob/main/LICENSE.md , https://research.meta.ai/blog/muse-spark-open

세션/영상

- https://developers.meta.com/resources/videos/building-with-muse-spark-and-muse-code , https://developers.meta.com/resources/videos/multimodal-intelligence-of-muse-spark , https://developers.meta.com/resources/videos/the-road-ahead-superintelligence-blueprint , https://dev.meta.ai/resources/videos/

파트너

- https://www.baseten.co/ , https://www.baseten.co/library/muse-spark-open/
- https://modal.com/ , https://modal.com/blog/meta-muse-spark-open-now-available-on-modal , https://modal.com/library/meta/muse-spark-open , https://modal.com/docs/guide/endpoints , https://modal.com/docs/guide/gpu , https://modal.com/docs/guide/sandboxes , https://modal.com/docs/guide/training-quickstart
- https://fireworks.ai/ , https://fireworks.ai/models/fireworks/muse-spark-open , https://fireworks.ai/models/fireworks/muse-glimmer-30b , https://fireworks.ai/blog/meta-muse-glimmer , https://docs.fireworks.ai/fine-tuning/training-api/serverless , https://github.com/fw-ai/cookbook , https://www.linkedin.com/in/sinan-ozdemir/
- https://www.together.ai/models/muse-glimmer , https://www.together.ai/models/muse-spark-open , https://docs.together.ai/docs/post-training/quickstart-sft , https://docs.together.ai/docs/post-training/quickstart-rl , https://github.com/togethercomputer/post-training-cookbook
- https://console.groq.com/docs/model/meta-models/muse-spark-open , https://openrouter.ai/meta/muse-spark-open , https://recipes.vllm.ai/meta-models/Muse-Spark-Open-816B-A42B , https://recipes.vllm.ai/meta-models/Muse-Glimmer-30B , https://docs.sglang.io/cookbook/autoregressive/Meta/MuseSparkOpen , https://pytorch.org/blog/muse-spark-open , https://github.com/huggingface/trl/tree/main/examples/sft_muse_spark , https://unsloth.ai/docs/models/muse-spark-open

과거 해커톤 / 참가작

- https://ai.meta.com/blog/llamacon-hackathon/ , https://ai.meta.com/blog/llama-3-hackathon-recap-cerebral-valley/ , https://about.fb.com/news/2024/11/metas-llama-impact-hackathon-pioneering-ai-solutions-for-public-good/ , https://metallama3.devpost.com/project-gallery , https://www.youtube.com/playlist?list=PLb0IAmt7-GS2eX0SYxHXdKPDwTNUxXB15
- https://github.com/TheCl3m/llama-hack , https://github.com/michaelwaves/llamacon-hackathon , https://github.com/torayeff/llamacon-hackathon-2025-sf/ , https://github.com/williamjsdavis/geo-lm , https://github.com/BasedHardware/openglass
- https://github.com/SaintSinC/pocket-muse , https://github.com/Unclefole/yebo-muse-agent , https://github.com/ztdgtll/musez-home , https://github.com/kopias/sous

---

건너뛴 것: X/Threads 원문 직접 열람(403), Discord #resources 녹화, 로그인 필요한 CV 참가자 가이드 원본, Baseten `-1026` ID의 Baseten 측 공식 확인. 리스크: 심사 가중치·세션 일정·Fireworks 크레딧 액수($100 vs $25)는 서드파티 전사본에서 나온 것이라 공식 가이드와 대조해야 한다.